

# UML

## Введение в UML (Unified Modeling Language)

UML (Unified Modeling Language) представляет собой стандартизированный язык визуализации, разработанный для описания, проектирования и документирования систем программного обеспечения. В отличие от конкретных языков программирования, UML обеспечивает способ моделирования поведения и структуры системы, что делает его универсальным инструментом для проектирования сложных систем независимо от того, на какой платформе или в каком языке разработки будет реализована система.

**Основная цель UML** — предоставить общий язык для разработчиков и других участников проектных команд (например, аналитиков, заказчиков), позволяя всем визуализировать архитектуру и процессы на понятном для них уровне абстракции. Это способствует более четкому пониманию системы и упрощает ее последующее проектирование и реализацию.

## История и эволюция UML

UML был разработан в 1990-х годах в ответ на растущие потребности в стандартизированной методологии проектирования сложных систем. В процессе разработки UML объединил несколько ранее существовавших методов и подходов к моделированию, таких как методики OMT (Object Modeling Technique) и Booch Method. В 1997 году UML был принят в качестве стандарта организацией Object Management Group (OMG), что окончательно утвердило его как основной инструмент в области объектно-ориентированного анализа и проектирования.

## Основные элементы UML

UML включает в себя три основных типа диаграмм:

1. **Структурные диаграммы.** показывают статическую структуру системы. Примеры: диаграммы классов, объектов, компонентов, пакетов, развертывания и композитной структуры.
2. **Диаграммы поведения.** отражают динамику системы, включая взаимодействие объектов. Примеры: диаграммы прецедентов (Use Case), активности, состояний и конечных автоматов.
3. **Диаграммы взаимодействий.** детализируют взаимодействие между объектами. Примеры: диаграммы последовательностей, коммуникации, времени и обзора взаимодействий.

UML поддерживает работу как на макроуровне (весь проект в целом), так и на микроуровне (детализация отдельных компонентов системы). Это делает его особенно полезным для документирования и проектирования систем с высокой степенью сложности, включая клиент-серверные архитектуры, распределенные системы и корпоративные приложения.

## Зачем использовать UML?

1. **Визуализация системы.** UML позволяет представлять структуру и поведение системы на различных уровнях абстракции, что упрощает понимание системы всеми участниками проекта.
2. **Документирование.** UML облегчает документирование решений и архитектуры системы, что особенно полезно при передаче проекта или его дальнейшей поддержке и масштабировании.
3. **Упрощение проектирования.** Использование UML помогает выявить архитектурные проблемы на ранних стадиях проектирования и позволяет их оперативно решать.
4. **Улучшение коммуникации.** UML предоставляет единую базу для обсуждения и согласования решений между разработчиками, аналитиками, менеджерами и заказчиками.

## Роль UML в написании технической документации

## 1. GDD (Game Design Document):

- В **GDD** диаграммы прецедентов и активности используются для описания ключевых сценариев игрового процесса. Например, диаграммы прецедентов помогают показать взаимодействие игрока с игровыми элементами (задания, взаимодействие с NPC), а диаграммы активности описывают детализированные процессы выполнения игровых действий (например, поиск предметов, выполнение миссий).
- **Диаграммы состояний** также могут применяться для описания поведения игровых объектов, таких как персонажи или объекты, которые изменяют свои состояния в ответ на действия игрока.

## 2. SRS (Software Requirements Specification):

- **Диаграммы прецедентов** особенно важны для описания функциональных требований системы. Они помогают представить взаимодействие пользователей с системой через набор сценариев использования, что упрощает понимание требований как для разработчиков, так и для заказчика.
- **Диаграммы классов** в **SRS** могут быть использованы для описания основных сущностей системы и их взаимосвязей, что помогает установить архитектурные рамки и основные зависимости между компонентами системы. То есть без детальной или сверхдетальной проработки — рамочно.

## 3. SDD (Software Design Document) или TDD (Test-Driven Development):

- В **SDD** диаграммы классов, компонентов и развертывания играют ключевую роль в описании структуры системы. Диаграммы классов используются для детализации связей между сущностями системы, а диаграммы компонентов и развертывания помогают спроектировать физическое развертывание приложения на серверах и других устройствах.
- **Диаграммы последовательностей** применяются для описания взаимодействий между компонентами на различных уровнях системы, что помогает лучше понять динамическое поведение системы.

## Отличия между SDD и TDD в контексте UML

**SDD (Software Design Document)** — это документ, описывающий архитектуру и структуру системы. В SDD диаграммы UML, такие как диаграммы классов, компонентов и развертывания, играют ключевую роль для визуализации структурных аспектов системы. Эти диаграммы помогают детализировать связи между сущностями системы, а также проектировать физическое размещение компонентов системы на серверах и других устройствах. SDD ориентирован на описание и документирование логики и архитектуры системы на этапе проектирования.

**TDD (Technical Driven Design)**, с другой стороны, направлен на создание системы с акцентом на технические аспекты разработки. В контексте UML TDD может использовать диаграммы классов и компоненты для того, чтобы структурировать технические решения, включая детали реализации архитектурных паттернов и взаимосвязей между модулями системы. Основной упор в TDD делается на выбор оптимальных технологий и инструментов для реализации системы.

Часто SDD и TDD могут быть одним целым и просто называться SDD или TDD, но могут быть и отдельными документами.

UML серьезно помогает в процессе разработки программного обеспечения, начиная с этапа анализа требований и заканчивая этапами тестирования и развертывания.

## Основные типы UML-диаграмм

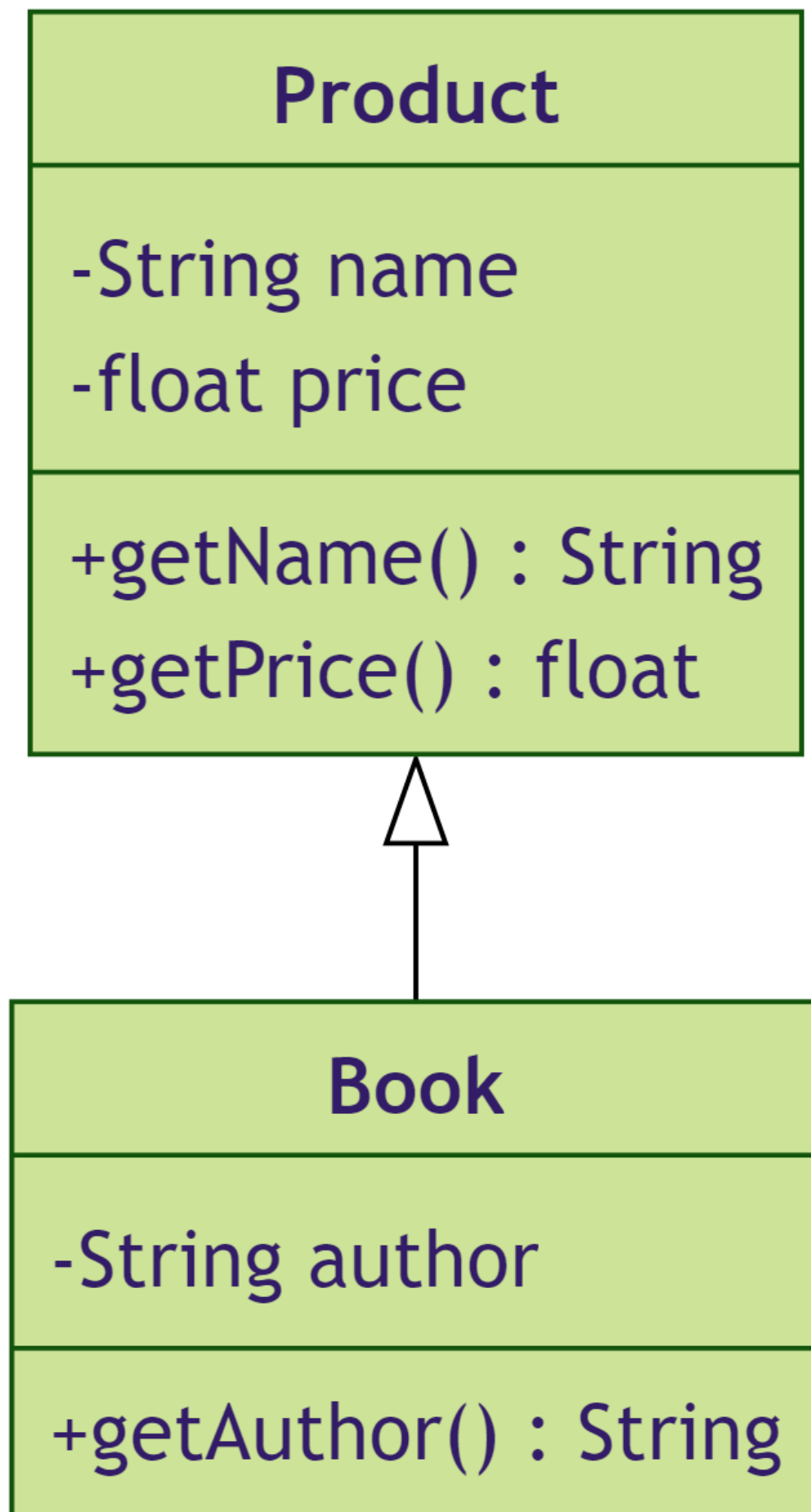
UML включает в себя три основных типа диаграмм: структурные, поведенческие и диаграммы взаимодействий. Эти диаграммы используются для моделирования различных аспектов системы на разных уровнях детализации.

## 1. Структурные диаграммы

Эти диаграммы показывают статическую структуру системы, включая классы и физические компоненты. Основная цель структурных диаграмм — зафиксировать архитектуру и структуру системы.

- **Диаграмма классов**

Визуализирует основные элементы системы: классы, их атрибуты, методы и отношения между ними. Это ключевая диаграмма для моделирования структуры системы. Пример:



В данном примере класс Book наследует свойства и методы от класса Product.

Ссылки: [EdrawMax](#), [Visual Paradigm](#), [Wikipedia - EN](#), [Wikipedia - RU](#)

- **Диаграмма развертывания**

Моделирует физическую архитектуру системы, показывая, на каких узлах или устройствах размещены программные компоненты.

Ссылки: [EdrawMax](#), [Visual Paradigm](#), [Wikipedia - EN](#), [Wikipedia - RU](#)

## 2. Диаграммы поведения

Эти диаграммы моделируют динамические аспекты системы, включая взаимодействие объектов и процессы внутри системы.

- **Диаграмма прецедентов (Use Case Diagram)**

Визуализирует взаимодействие пользователей с системой через набор прецедентов, однако эта диаграмма не поддерживается в Mermaid. Ссылки: [Lucidchart](#), [EdrawMax](#), [Wikipedia - EN](#), [Wikipedia - RU](#)

- **Диаграмма активности**

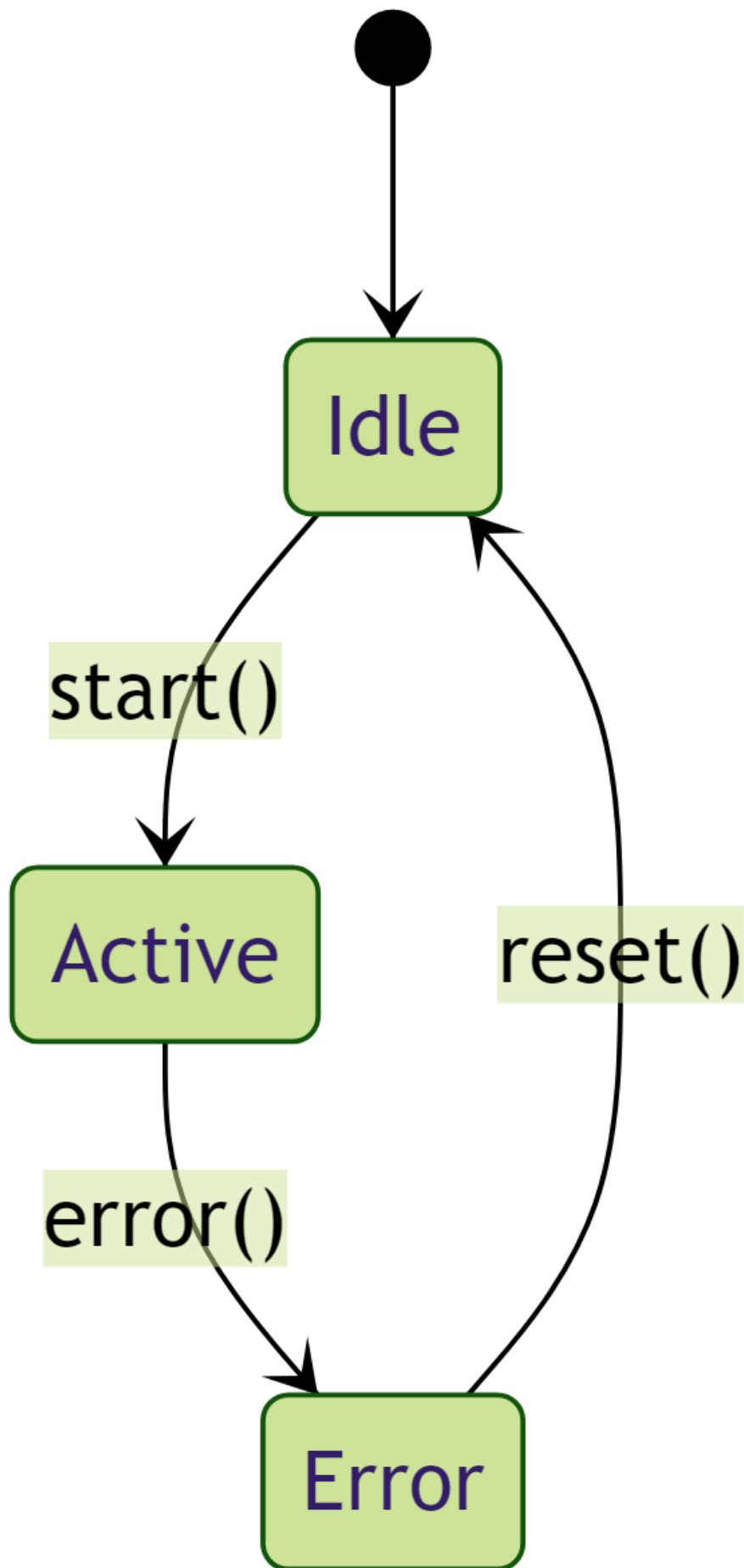
Моделирует последовательность действий в системе, включая потоки управления и данных. Это полезно для описания бизнес-процессов и рабочих потоков.

Ссылки: [Lucidchart](#), [EdrawMax](#), [Wikipedia - EN](#), [Wikipedia - RU](#)

- **Диаграмма состояний**

Моделирует поведение объекта через его возможные состояния и переходы между ними в ответ на события.

Пример:



В этом примере объект переходит из состояния “Idle” в “Active” при вызове метода `start()`, и может вернуться в состояние “Idle” после ошибки или сброса.

Ссылки: [EdrawMax](#), [Lucidchart](#), [Wikipedia - EN](#), [Wikipedia - RU](#)

- **Диаграмма конечных автоматов (State Machine Diagram)**

Диаграмма конечных автоматов детализирует состояния объекта и переходы между ними в рамках реактивных систем, где объект изменяет состояние в ответ на внешние или внутренние события.

Ссылки: [EdrawMax](#), [Visual Paradigm](#), [Wikipedia - EN](#), [Wikipedia - RU](#), [Диаграмма конечного автомата и](#)

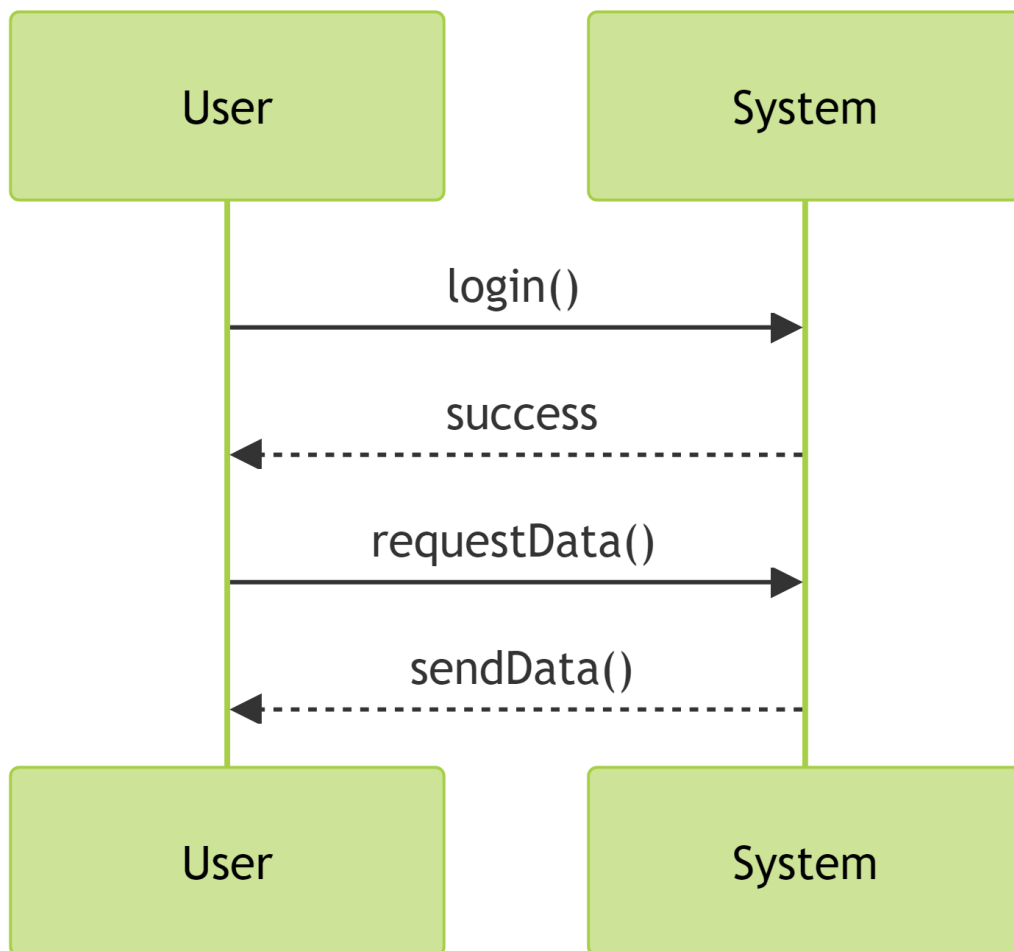
### 3. Диаграммы взаимодействий

Эти диаграммы моделируют процесс обмена сообщениями между объектами во времени, что помогает описывать динамическое поведение системы.

- **Диаграмма последовательностей**

Отображает взаимодействие объектов в хронологическом порядке, акцентируя внимание на последовательности сообщений между ними.

Пример:



В этом примере показано взаимодействие между пользователем и системой с обменом сообщениями.

Ссылки: [Lucidchart](#), [EdrawMax](#), [Wikipedia - EN](#), [Wikipedia - RU](#)

- **Диаграмма коммуникации**

Показывает связи между объектами и передаваемые между ними сообщения.

- Ссылки: [Visual Paradigm](#), [EdrawMax](#)

Эти диаграммы помогают эффективно моделировать как статическую структуру системы, так и её динамические процессы, предоставляя проектировщикам более точное представление о всех аспектах системы.

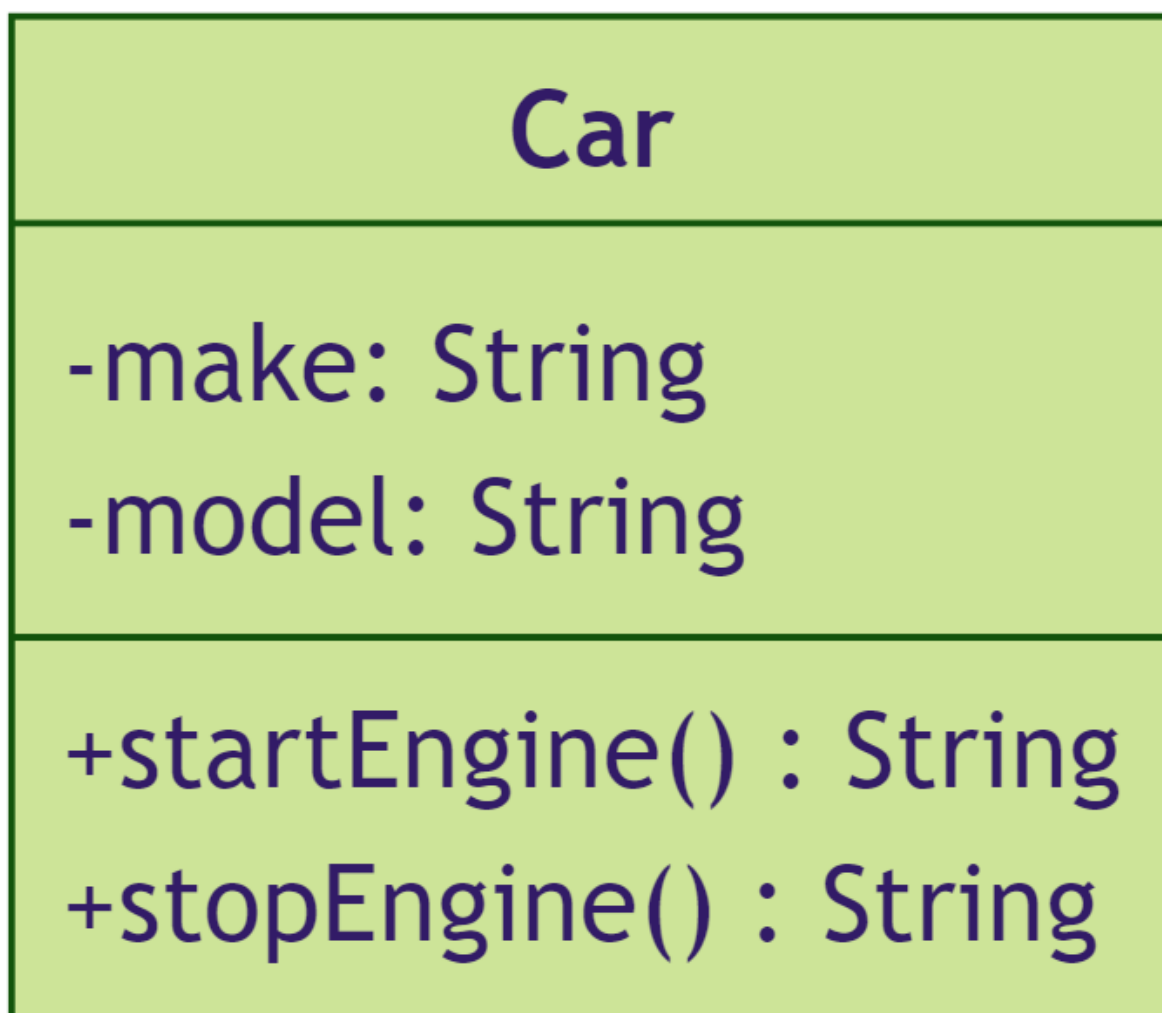
## Диаграммы классов

Диаграмма классов UML является центральным элементом для визуализации и проектирования структуры объектно-ориентированных систем. Она используется для отображения классов, их атрибутов, методов, а также взаимоотношений между ними. Каждый элемент диаграммы классов имеет своё значение и обозначение, позволяя точно моделировать объекты и их взаимодействия в системе.

### Основные элементы диаграммы классов:

1. **Класс.** Представляет собой шаблон, по которому создаются объекты. В UML класс обозначается прямоугольником, который разделён на три части:
  - Верхняя часть содержит **имя класса**.
  - Средняя часть содержит **атрибуты класса**.
  - Нижняя часть — **методы класса**.

Пример:

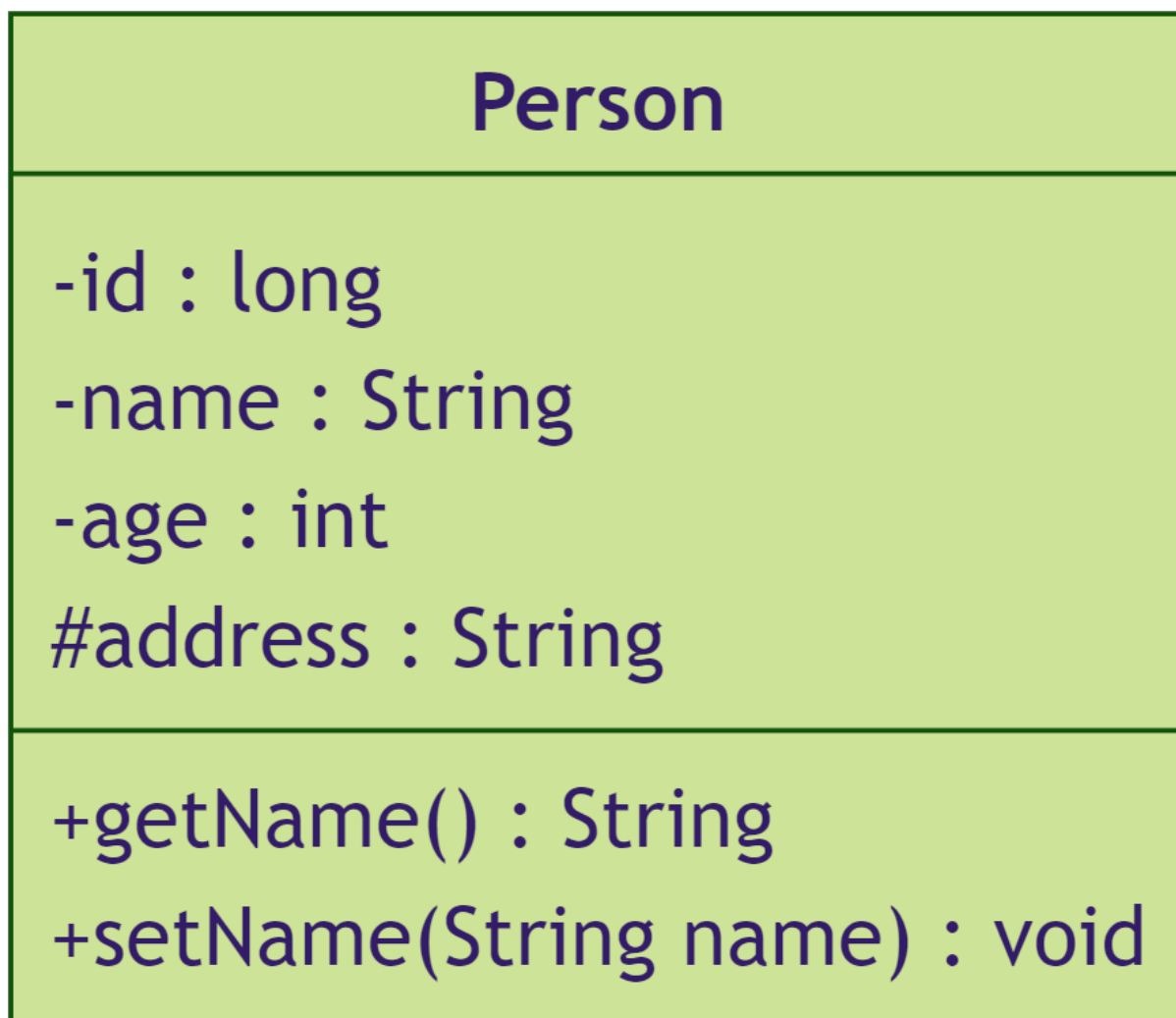


Здесь класс Car имеет два атрибута make и model (марка и модель), а также два метода: startEngine() и stopEngine().

2. **Атрибуты.** Представляют собой свойства класса. Атрибуты могут иметь модификаторы доступа:
  - **Private** (-): Атрибут доступен только внутри класса.
  - **Protected** (#): Атрибут доступен внутри класса и его подклассов.

- **Public (+):** Атрибут доступен всем.

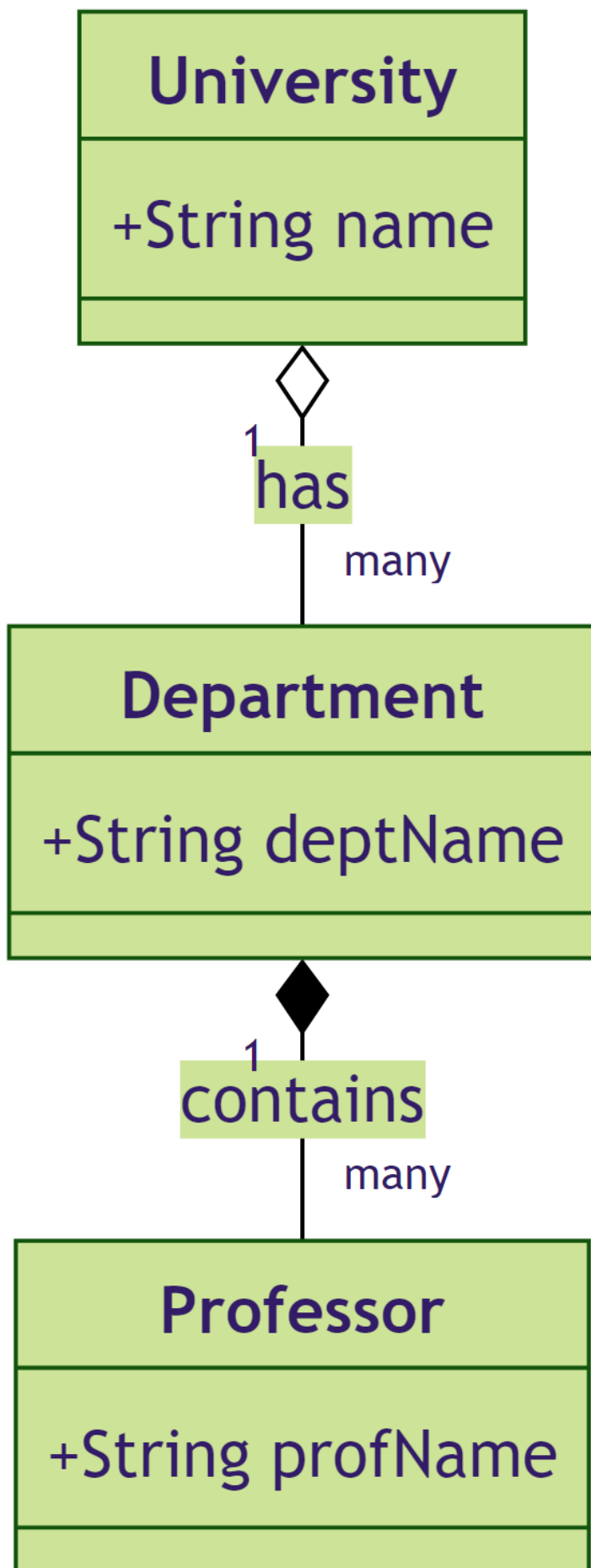
Пример:



В классе `Person` атрибуты `name` и `age` являются приватными, а `address` — защищённым.

3. **Методы.** Методы описывают поведение класса. Как и атрибуты, они могут быть с различными уровнями доступа (`private`, `protected`, `public`).
4. **Отношения между классами:**
  - **Ассоциация.** Линия между двумя классами, обозначающая связь. Ассоциации могут иметь стрелки для указания направления.
  - **Агрегация** (нежёсткая связь): Пустой ромб на стороне класса-владельца указывает на то, что один класс содержит другой, но не контролирует его жизненный цикл.
  - **Композиция** (жёсткая связь): Закрашенный ромб указывает на более сильную связь, где класс-владелец контролирует жизненный цикл содержащегося объекта.

Пример агрегации и композиции:

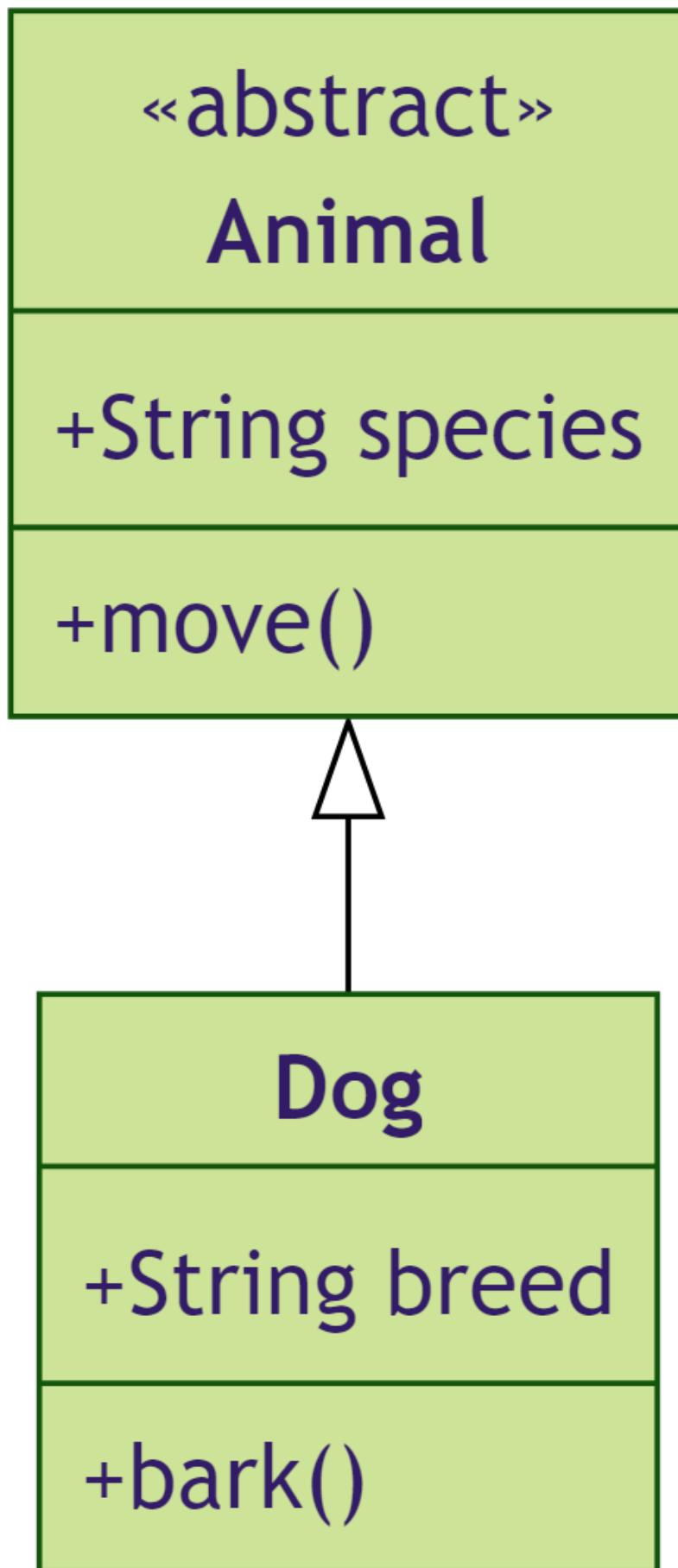


---

Здесь University агрегирует Department, а Department композит Professor.

5. **Наследование.** Показывает, что один класс является производным от другого (суперкласс — подкласс). Обозначается сплошной линией с треугольной стрелкой, указывающей на суперкласс.

Пример:

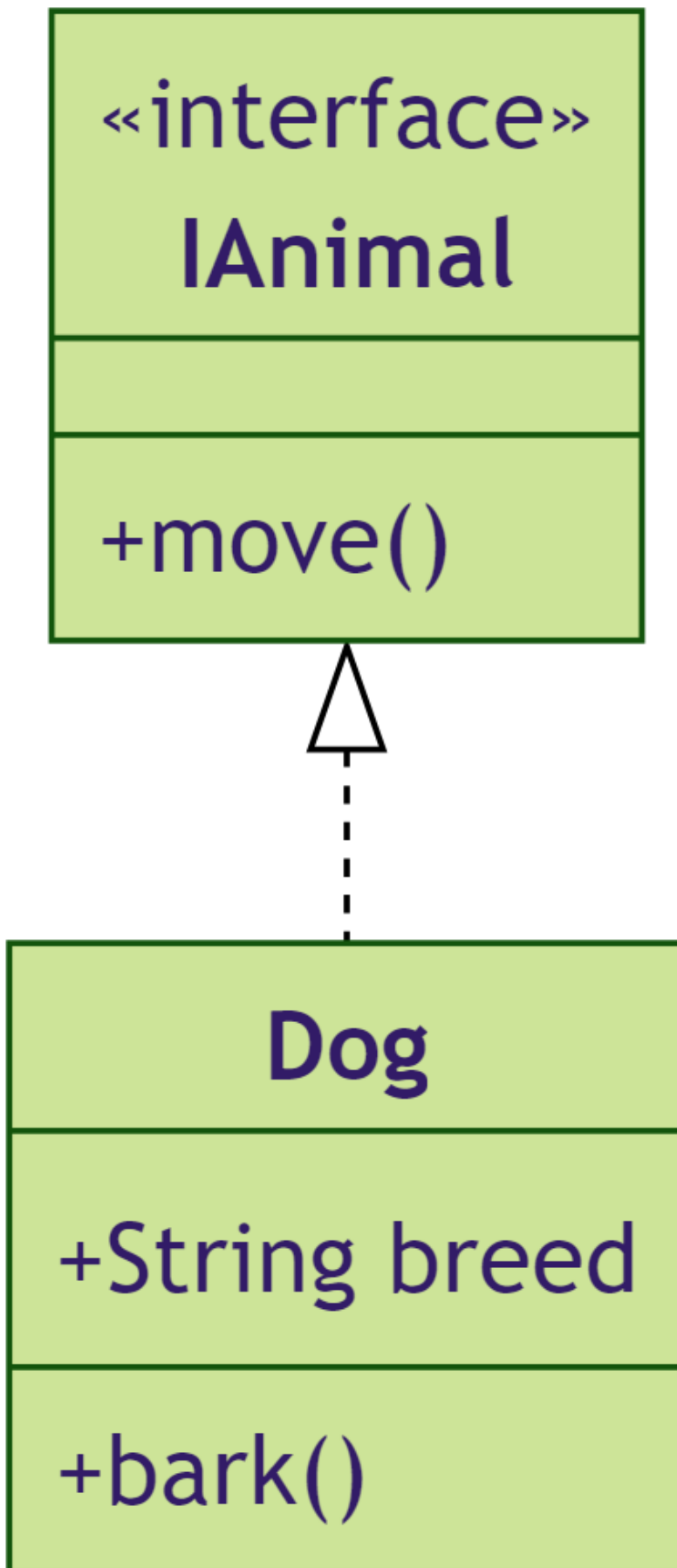


---

Класс Dog наследует все свойства и методы класса Animal.

6. **Стереотипы.** В UML стереотипы добавляются в кавычках над классом для обозначения его особого типа, например, <<interface>> для интерфейсов.

Пример интерфейса:



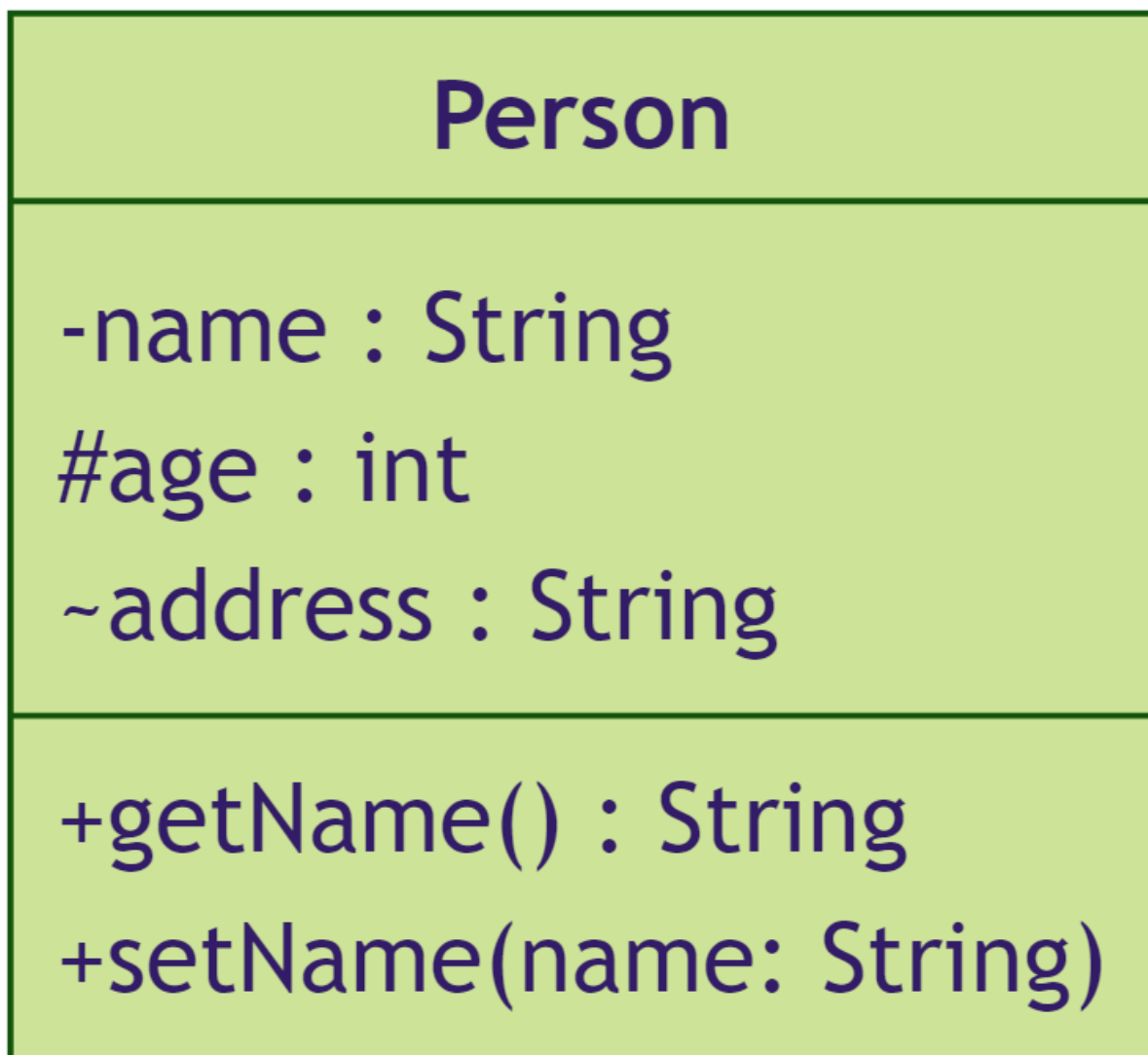
Интерфейс `IAntimal` реализуется классом `Dog`.

## Подробности обозначений и отношений в диаграммах классов UML

### Модификаторы доступа

- **Private (-)**. Атрибуты и методы, помеченные как `private`, могут использоваться только внутри данного класса.
- **Protected (#)**. Доступны в пределах класса и его наследников.
- **Public (+)**. Открытые атрибуты и методы доступны всем.
- **Package (~)**. Специфичный для пакета уровень доступа, доступен только для классов внутри одного пакета.

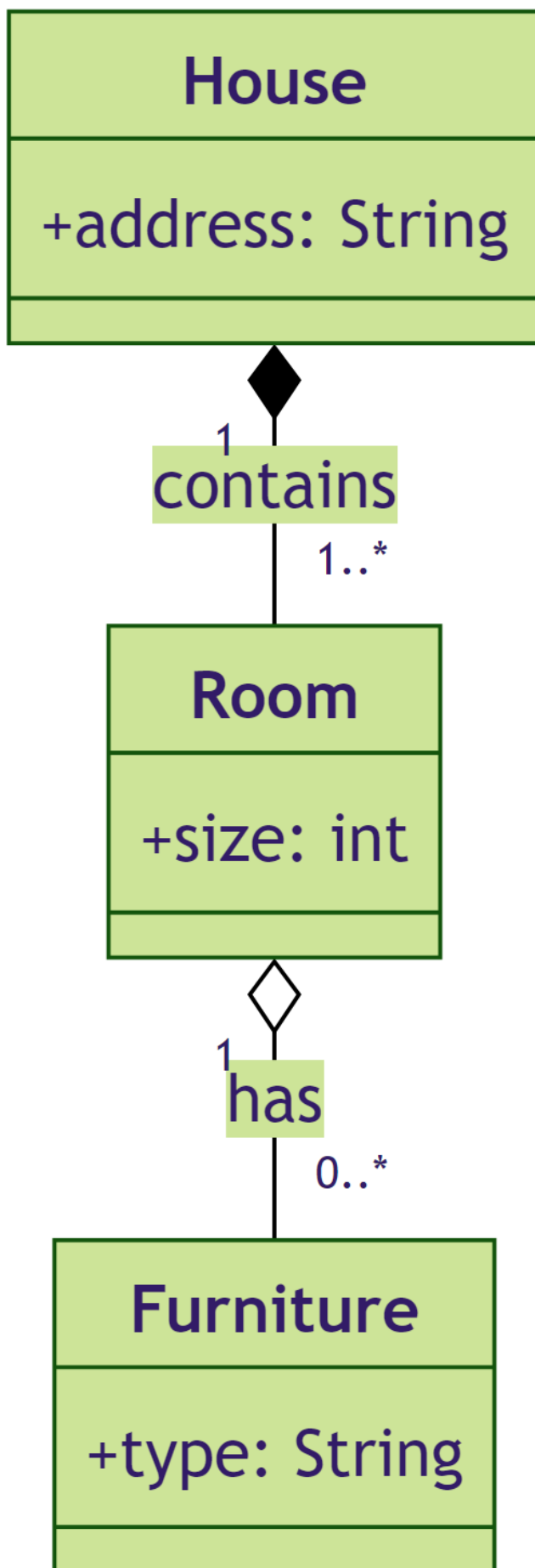
Пример с модификаторами доступа:



### Композиция и агрегация

- **Композиция** (жёсткая связь): Если класс `A` композит класс `B`, то уничтожение класса `A` приводит к уничтожению класса `B`. Это связь “часть-целое”. Обозначается чёрным ромбом на стороне класса-целого.
- **Агрегация** (слабая связь): Класс `A` агрегирует класс `B`, но `B` может существовать независимо. Обозначается пустым ромбом на стороне класса-целого.

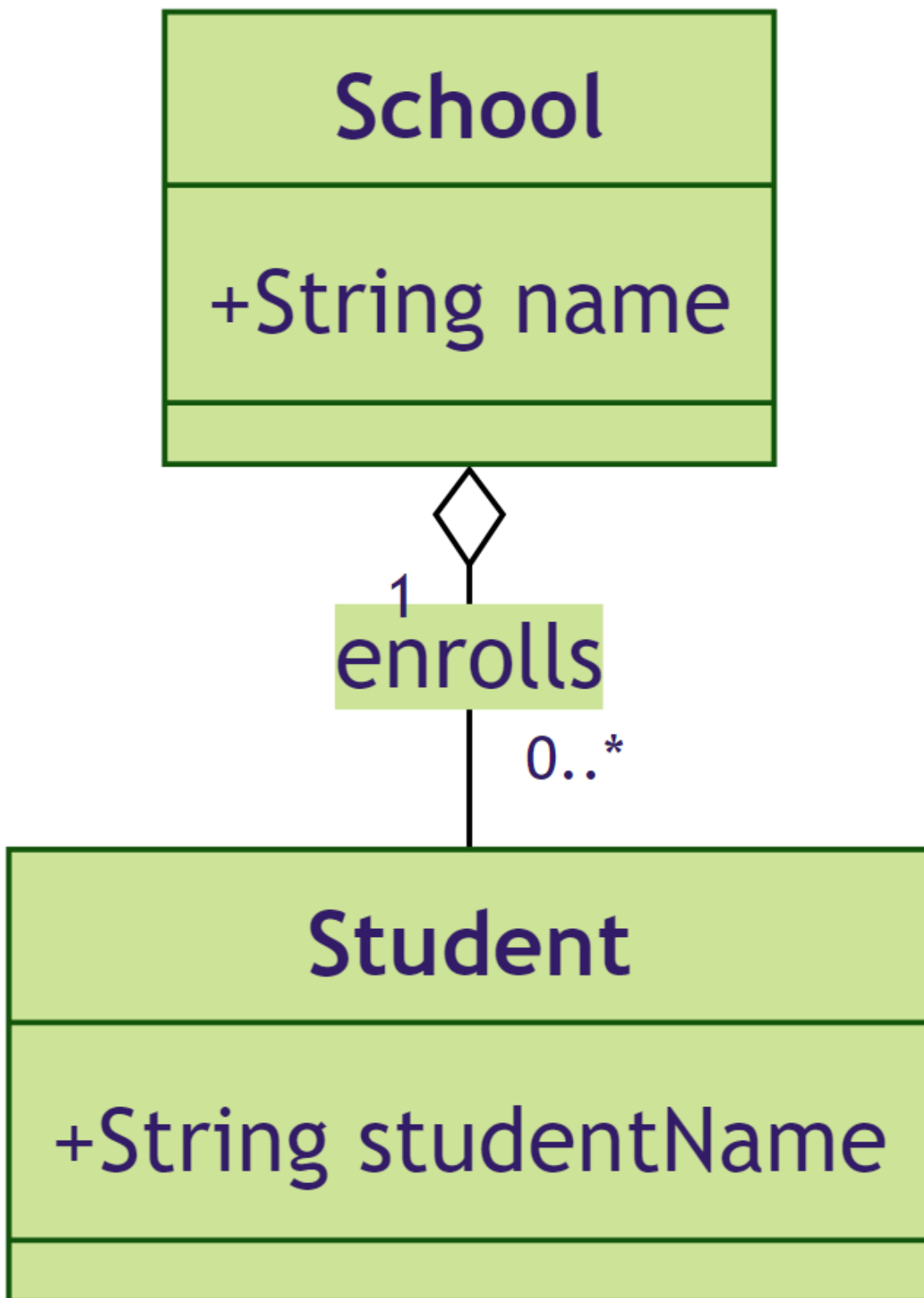
Пример композиции и агрегации:



## Кардинальность (cardinality, multiplicity)

Кардинальность указывает на количество объектов, связанных с другим объектом. Например: - 1: Ровно один объект.  
- 0..1: Ноль или один объект. - \* или 0..\*: Ноль или больше объектов. - 1..\*: Один или больше объектов.

Пример кардинальности:

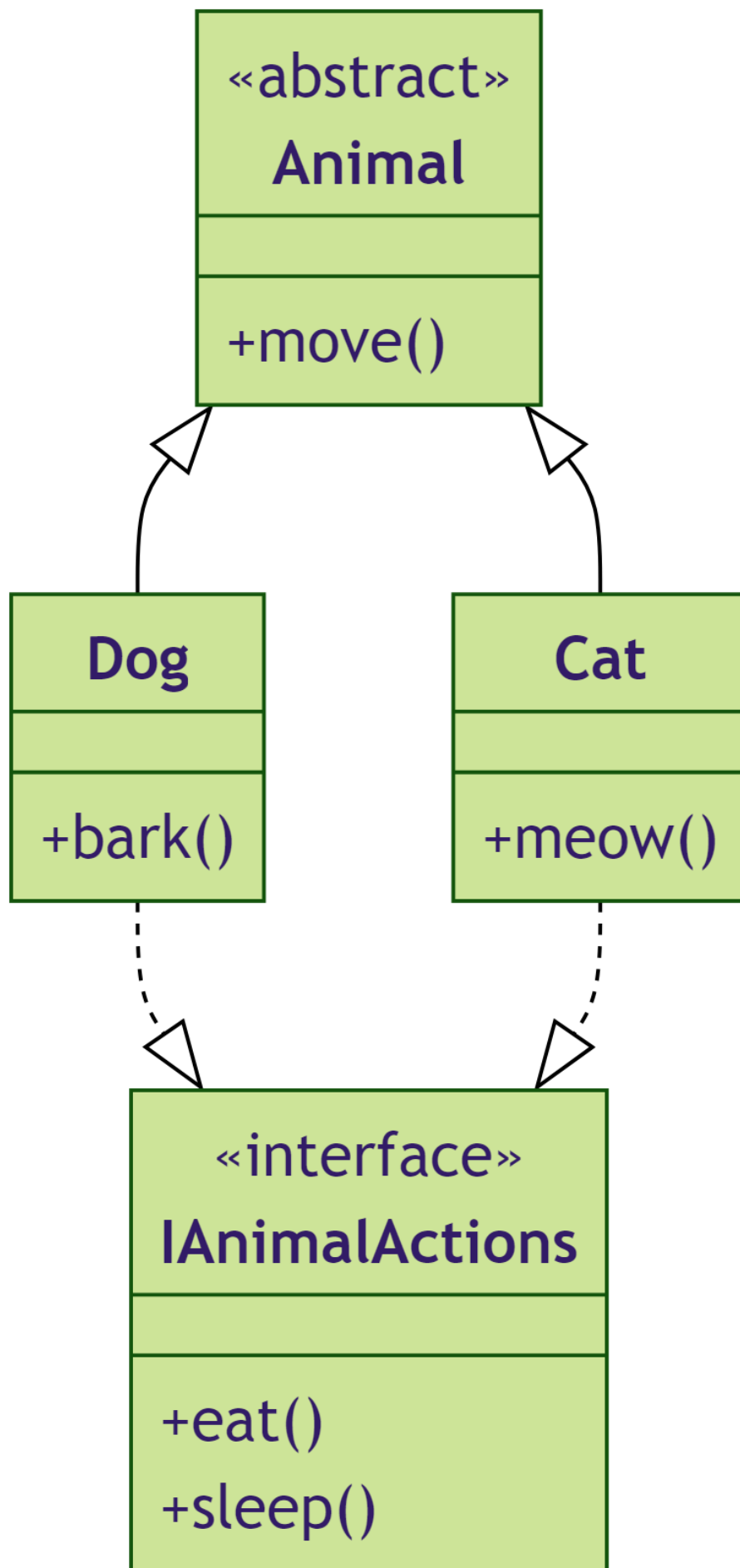


### Наследование и реализация

- **Наследование.** Один класс наследует свойства и методы другого класса (суперкласса). Обозначается сплошной стрелкой с треугольной головой.

- **Реализация.** Класс реализует интерфейс, что означает, что он должен предоставить конкретные реализации для всех методов интерфейса. Обозначается пунктирной стрелкой с треугольной головой.

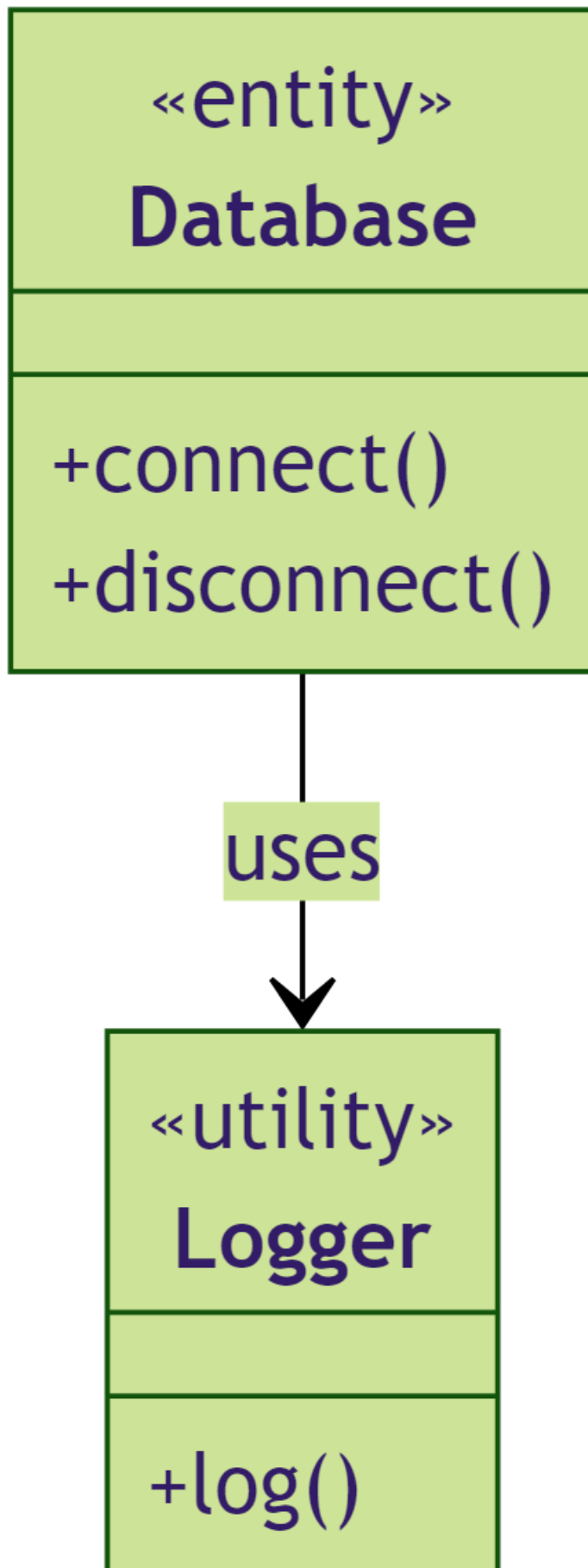
Пример наследования и реализации:



## Стереотипы

Стереотипы в UML добавляют дополнительную информацию о классах и отношениях. Например, `<<interface>>` используется для обозначения интерфейса, `<<entity>>` — для сущности в системе. Также стереотипы могут применяться к отношениям, например, для обозначения специфических связей между классами.

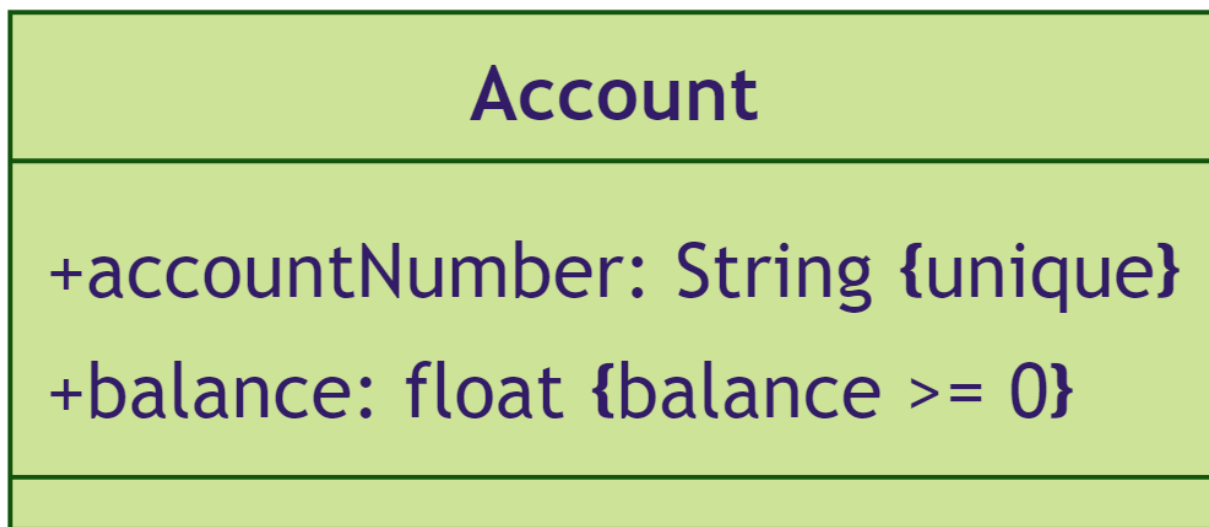
Пример стереотипов:



## Ограничения (constraints)

Ограничения позволяют задать правила или условия для элементов на диаграмме классов. Они заключаются в фигурные скобки {} и указывают, например, на обязательность или уникальность атрибутов.

Пример ограничений:



Эти концепции расширяют базовые возможности диаграмм классов, делая их более мощными и точными для описания сложных систем.

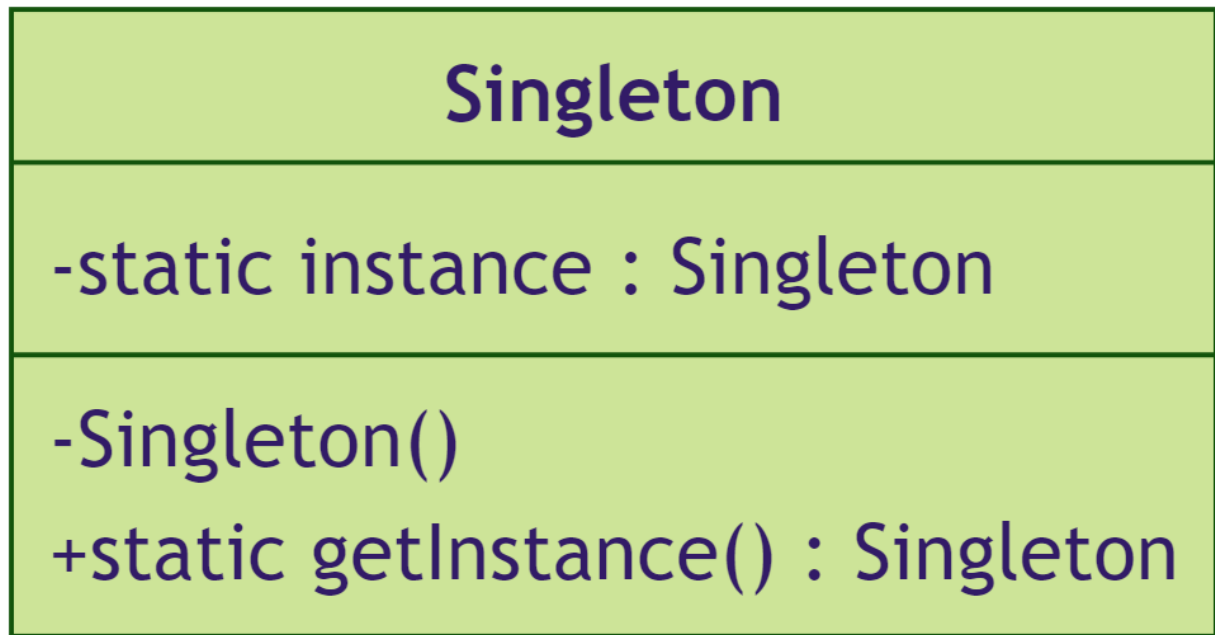
## Применение диаграмм классов в SRS и SDD

- В **SRS** диаграммы классов используются для моделирования основных сущностей системы и их взаимосвязей, что помогает определить ключевые компоненты и их взаимодействие.
- В **SDD** диаграммы классов более детализированы и включают архитектурные компоненты системы, их интерфейсы, связи и особенности реализации, что позволяет инженерам программного обеспечения точно представить архитектуру системы и её компонентов.

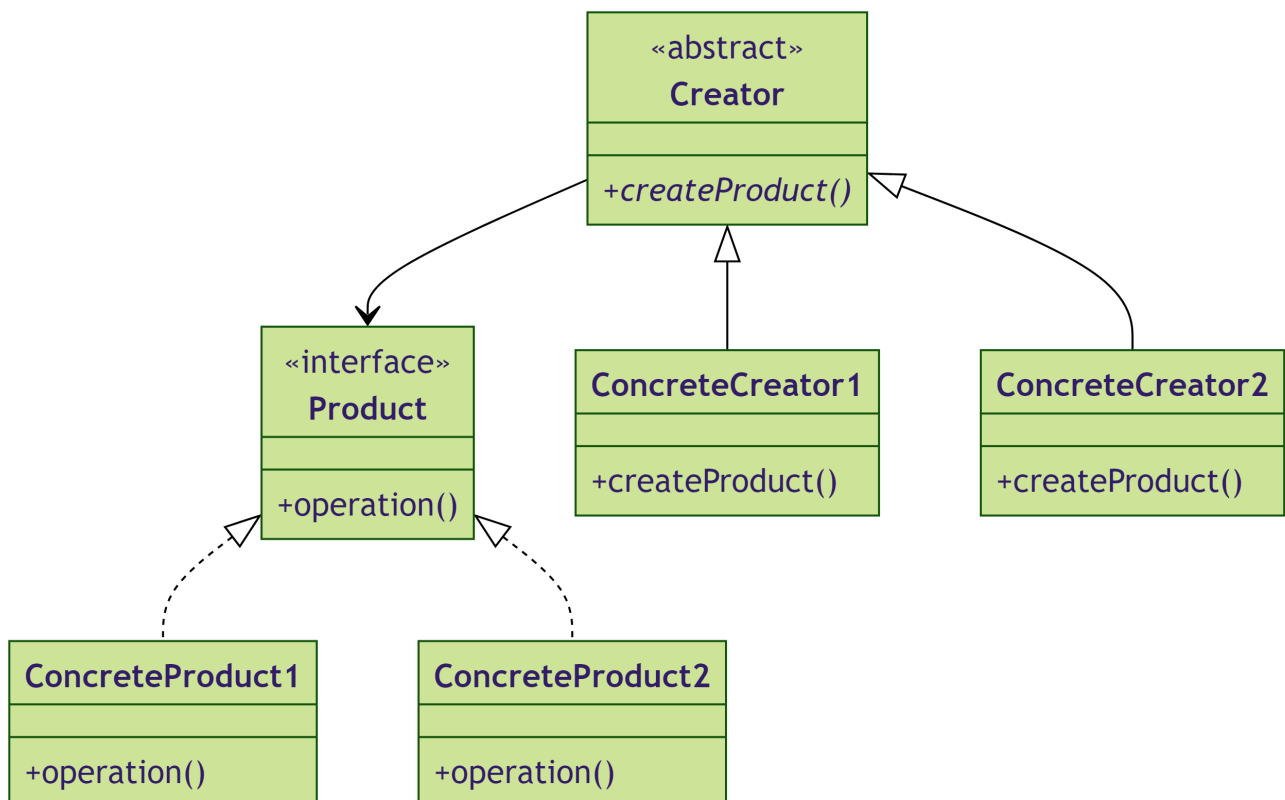
Диаграммы классов обеспечивают визуальное понимание структуры системы и позволяют моделировать объектные отношения, что критично для проектирования программных систем любого масштаба.

## Примеры паттернов проектирования на диаграммах классов

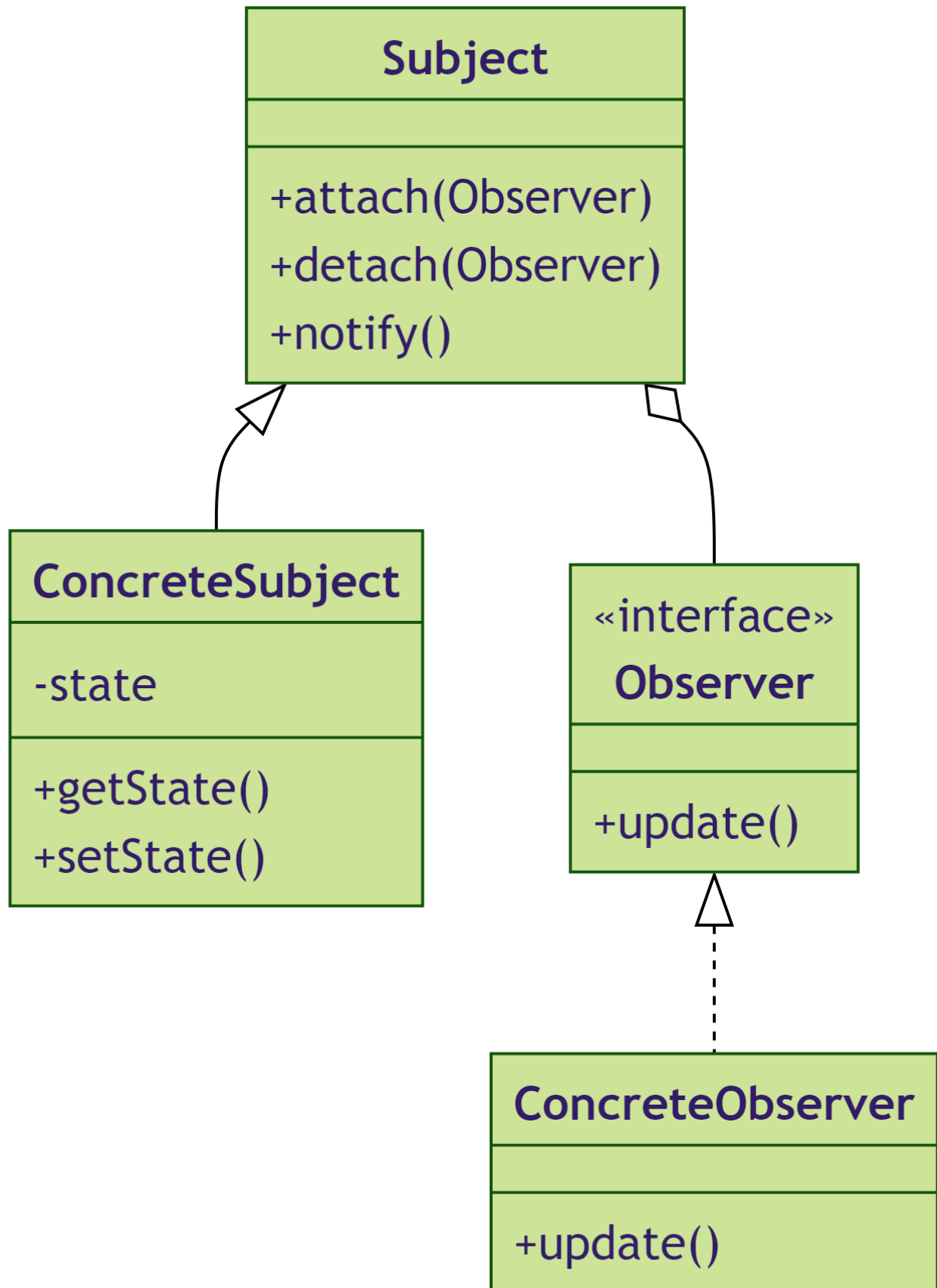
1. **Singleton (Одиночка)**: Паттерн Singleton гарантирует, что у класса будет только один экземпляр, и предоставляет глобальную точку доступа к этому экземпляру.



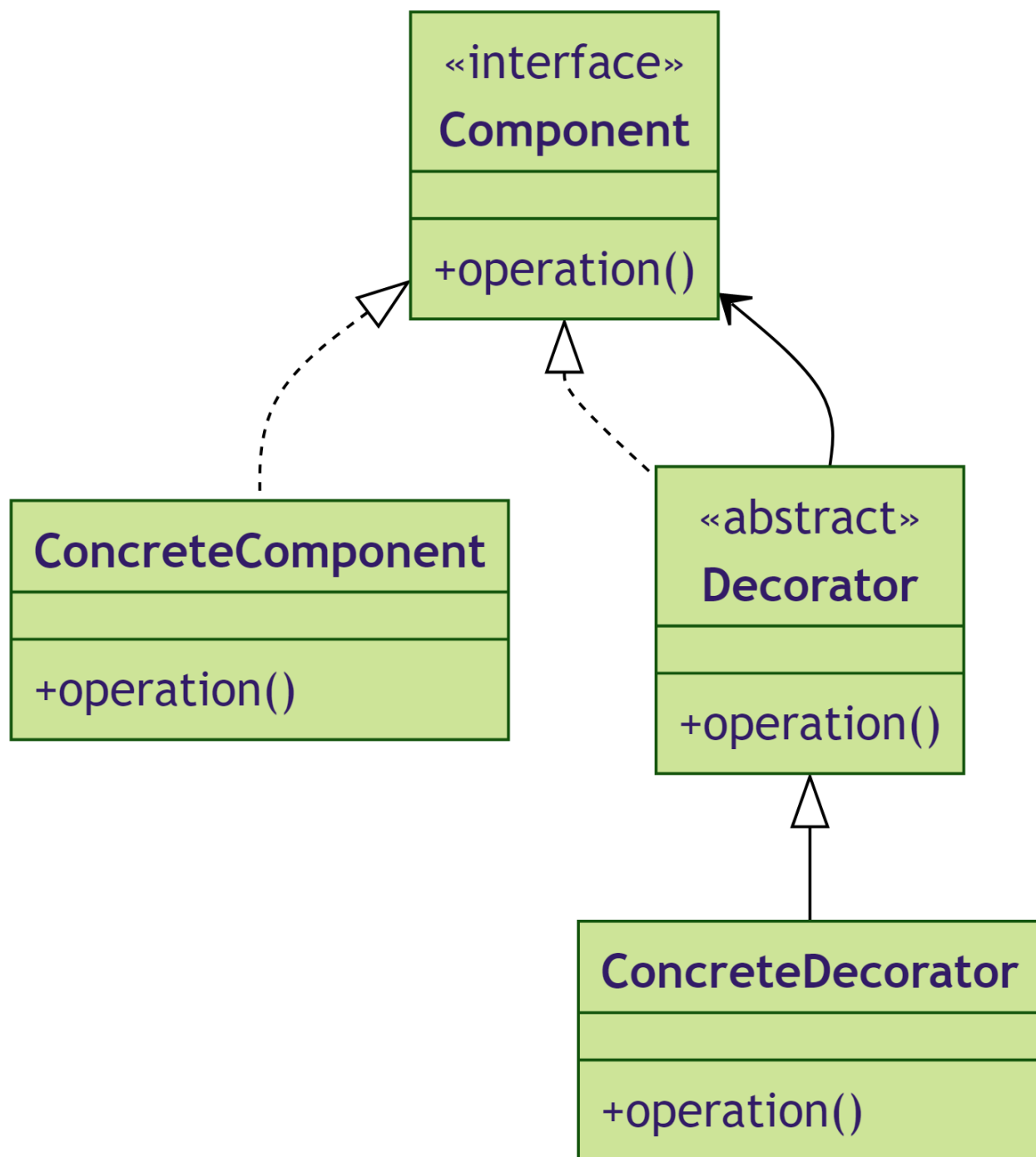
2. **Factory Method (Фабричный метод):** Этот паттерн предоставляет интерфейс для создания объектов в суперклассе, но позволяет подклассам изменять тип создаваемого объекта.



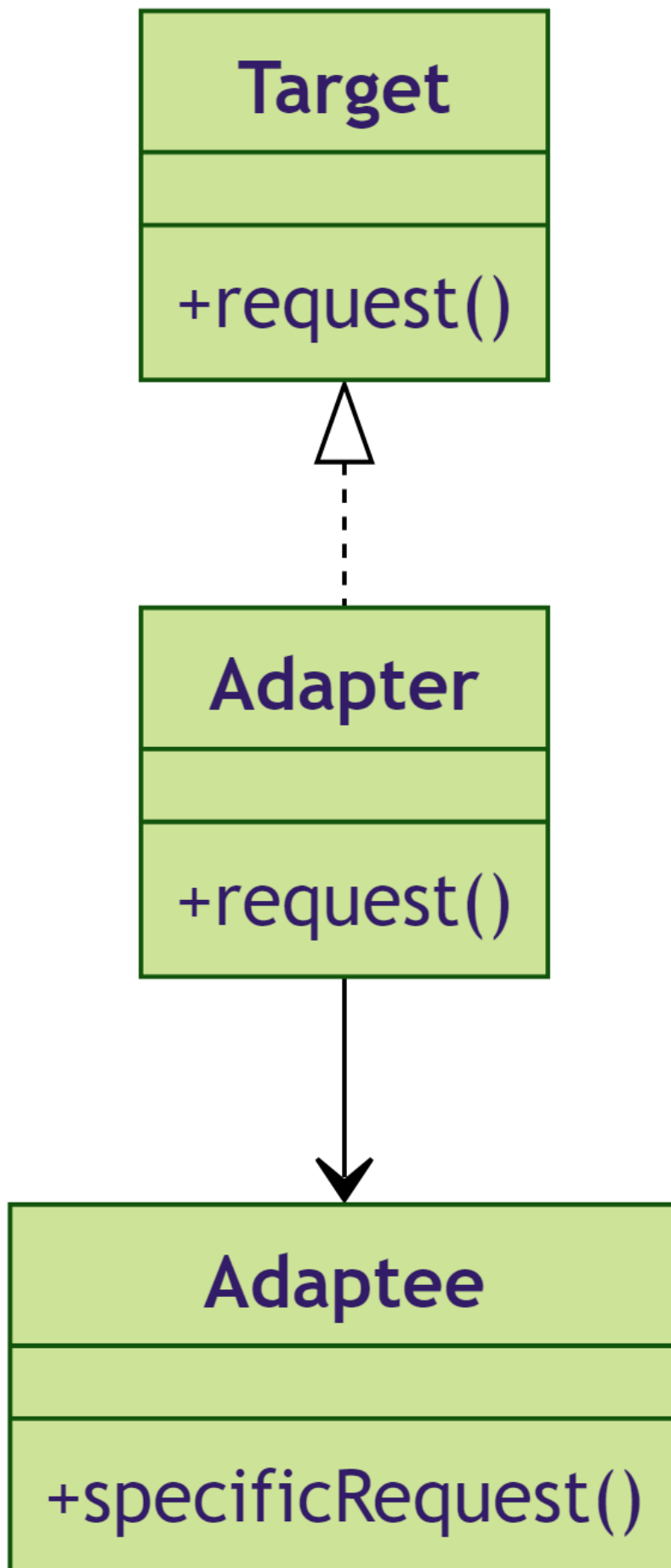
3. **Observer (Наблюдатель):** Этот паттерн определяет зависимость “один ко многим”, так что при изменении состояния одного объекта все зависимые объекты оповещаются и обновляются автоматически.



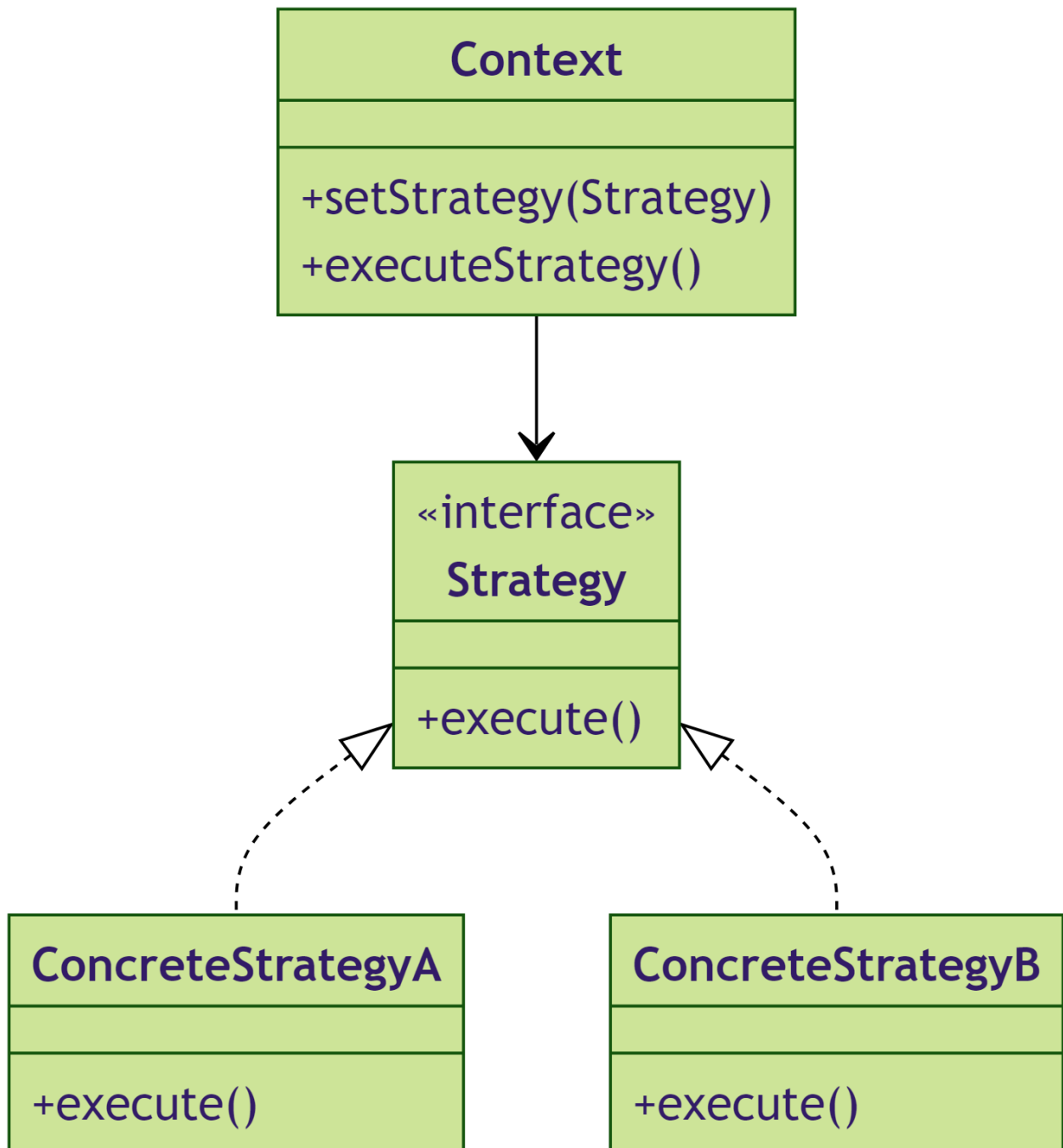
4. **Decorator (Декоратор)**: Паттерн Decorator позволяет динамически добавлять объектам новые обязанности, обобщая их в специальные классы-декораторы.



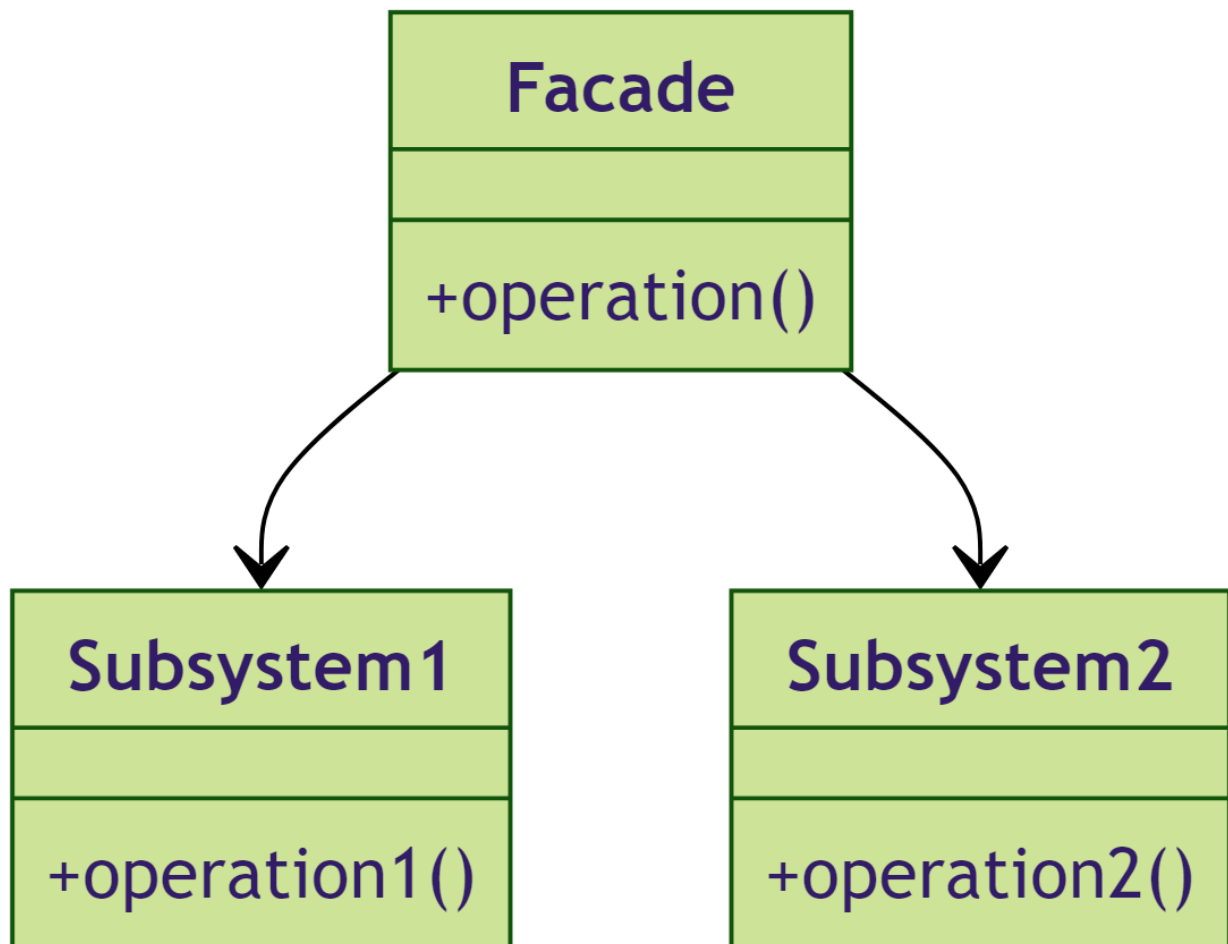
5. **Adapter (Адаптер)**: Паттерн Adapter позволяет объектам с несовместимыми интерфейсами работать вместе, преобразуя интерфейс одного объекта в понятный другому объекту.



6. **Strategy (Стратегия):** Паттерн Strategy определяет семейство алгоритмов, инкапсулирует каждый из них и делает их взаимозаменяемыми. Это позволяет изменять поведение объекта без изменения его кода.



7. **Facade (Фасад):** Паттерн Facade предоставляет простой интерфейс для сложной подсистемы классов, облегчая взаимодействие с ней.



## Диаграммы прецедентов (Use Case Diagrams)

### Определение и назначение диаграммы вариантов использования

Диаграмма прецедентов (или диаграмма вариантов использования) — это поведенческая диаграмма UML, которая моделирует взаимодействие пользователей (акторов) с системой через сценарии использования (прецеденты). Эта диаграмма фокусируется на внешнем представлении системы и описывает, **какие функции или действия может выполнять система**, а также **кто взаимодействует с системой**.

Основная цель диаграммы прецедентов — описать **функциональные требования** системы с точки зрения её пользователей. Она полезна как на этапе анализа требований, так и на этапе проектирования, помогая согласовать ожидания между разработчиками и заказчиками.

### Основные элементы диаграммы прецедентов

#### 1. Акторы:

- Представляют внешние сущности, которые взаимодействуют с системой. Это могут быть люди (конечные пользователи), другие системы или устройства. Акторы обозначаются фигурками человечков.
- Пример: пользователь системы, администратор, внешняя база данных.

#### 2. Прецеденты (Use Cases):

- Прецедент — это действие или набор действий, выполняемых системой в ответ на запрос акторов. Это может быть любой процесс, который имеет ценность для акторов.
- Прецеденты обозначаются эллипсами с текстом внутри, указывающим, что делает система.
- Пример: «Логин в систему», «Создание заказа», «Публикация сообщения».

### 3. Система:

- Область системы или её часть, которая моделируется. Система обозначается в виде прямоугольника, в котором размещены все прецеденты, а акторы находятся снаружи.
- Пример: модуль онлайн-магазина, игровое приложение.

### 4. Связи (Ассоциации):

- Связи между акторами и прецедентами показывают, кто выполняет действие или участвует в нём. Связи обозначаются сплошными линиями без стрелок.
- Пример: пользователь взаимодействует с прецедентом «Покупка товара».

### 5. Включение (<<include>>):

- Связь <<include>> указывает, что один прецедент обязательно включает в себя другой прецедент. Это говорит о том, что часть функционала всегда выполняется в рамках другого процесса.
- Пример: прецедент «Регистрация» включает в себя «Подтверждение email».

### 6. Расширение (<<extend>>):

- Связь <<extend>> используется для моделирования необязательных действий, которые могут выполняться в определённых условиях. Этот прецедент расширяет другой прецедент.
- Пример: прецедент «Завершение заказа» может быть расширён «Добавлением подарочной упаковки» в некоторых случаях.

### 7. Обобщение (Generalization):

- Связь обобщения используется для указания, что один актор или прецедент является специализированной версией другого. Она обозначается стрелкой с треугольной головкой.
- Пример: актор «Привилегированный пользователь» может быть обобщением актора «Пользователь».

## Связь диаграммы прецедентов с GDD и SRS

- В **GDD (Game Design Document)** диаграмма прецедентов используется для описания **основных игровых сценариев**. Например, для игры диаграмма может описывать действия, такие как «Создание уровня», «Выбор персонажа», «Победа над боссом», что помогает разработчикам и дизайнерам понимать взаимодействие игрока с системой.
- В **SRS (Software Requirements Specification)** диаграммы прецедентов полезны для описания **функциональных требований** системы, таких как «Регистрация пользователя», «Оплата заказа», «Поиск товаров». Это помогает структурировать требования и облегчить их понимание командой разработки.

## Пример диаграммы прецедентов для игрового приложения

Диаграмма прецедентов для простого игрового приложения может выглядеть следующим образом:

- **Акторы:** «Игрок», «Администратор».
- **Прецеденты:** «Создание персонажа», «Управление инвентарём», «Бой с врагом», «Завершение уровня», «Настройка системы».
- **Связи:**
  - Игрок создаёт персонажа и управляет инвентарём.
  - Администратор выполняет настройку системы.
  - Прецедент «Бой с врагом» расширяется до «Использование оружия».

## Примеры диаграмм и ссылки

### 1. Диаграммы прецедентов на EdrawMax

[EdrawMax](#) предлагает полезные примеры и шаблоны для диаграмм прецедентов, которые показывают, как моделировать взаимодействие пользователей с системой.

### 2. Примеры на Lucidchart

[Lucidchart](#) предоставляет подробные руководства и примеры диаграмм прецедентов, включая сценарии для бизнес-приложений и программного обеспечения.

### 3. Visual Paradigm

[Visual Paradigm](#) детально описывает все элементы диаграммы прецедентов с примерами и возможностью создания диаграмм онлайн.

### 4. Wikipedia

[Wikipedia](#) содержит базовую информацию о диаграммах прецедентов, их элементах и использовании в программной инженерии.

Диаграммы прецедентов являются мощным инструментом для описания функциональных требований и сценариев взаимодействия пользователей с системой. Они широко используются в документации, таких как GDD и SRS, для моделирования взаимодействий, как в игровых приложениях, так и в корпоративных системах.

## Диаграммы состояний

Диаграммы состояний в UML — это поведенческие диаграммы, используемые для моделирования состояний объектов и их изменений в ответ на события. Эти диаграммы отображают возможные состояния объекта, а также переходы между ними, позволяя визуализировать **жизненный цикл** объекта. Они играют важную роль в проектировании приложений, где объекты проходят через различные состояния в зависимости от выполняемых операций.

## Назначение диаграмм состояний

Диаграммы состояний используются для моделирования динамических аспектов системы, таких как: - **Поведение объектов во времени** — как объект реагирует на внешние и внутренние события. - **Изменение состояний** — переходы объекта из одного состояния в другое, вызванные действиями пользователя, внутренними процессами или взаимодействием с другими объектами. - **Планирование ответных действий** — что происходит в системе после события, как обрабатываются данные и запускаются процессы.

Диаграммы состояний необходимы, когда поведение объекта зависит от предшествующего состояния или изменений во времени.

## Основные элементы диаграмм состояний

### 1. Состояния (States):

- Представляют собой возможные состояния объекта. Обозначаются прямоугольником с закругленными углами, внутри которого написано название состояния.
- Пример: состояние «Idle» (ожидание), «Running» (выполнение), «Error» (ошибка).

### 2. Переходы (Transitions):

- Указывают на изменение состояния объекта в ответ на событие. Переход обозначается стрелкой от одного состояния к другому. Стрелка сопровождается обозначением события, которое вызвало переход.
- Пример: стрелка от «Idle» к «Running» с надписью `start()` означает, что вызов метода `start()` переводит объект в состояние «Running».

### 3. Начальное состояние (Initial State):

- Начальное состояние диаграммы обозначается заполненным черным кругом, с которого начинается поведение объекта. Это исходная точка системы.
- Пример: черный круг с исходным состоянием «Idle».

### 4. Конечное состояние (Final State):

- Конечное состояние обозначается кругом с обводкой и маленьким черным кругом внутри. Оно указывает на завершение жизненного цикла объекта.
- Пример: конечное состояние после успешного выполнения работы или выхода из системы.

### 5. События (Events):

- Событие — это действие или сигнал, который приводит к изменению состояния объекта. События обозначаются как текст на переходах между состояниями.
- Пример: `timeout`, `clickButton`, `start`.

### 6. Действия (Actions):

- Это операции, выполняемые объектом при переходе из одного состояния в другое. Действия могут сопровождаться как в начале перехода, так и после перехода.
- Пример: при переходе в состояние «Error» может выполняться действие `logError()`.

## Расширенные элементы и обозначения

### 1. Сложные состояния (Composite States):

- В сложных системах состояние может содержать вложенные состояния. Это полезно для моделирования систем с иерархическими состояниями, где одно состояние может включать несколько под-состояний.
- Пример: состояние «Order Processing» может содержать под-состояния «Payment» и «Shipping».

### 2. Псевдосостояния (Pseudostates):

- Служат для разветвления логики поведения объекта. Сюда относятся: **решение** (decision), **разветвление** (fork), **объединение** (join).
- Пример: при разветвлении состояние объекта может зависеть от результата проверки (`if-else`).

### 3. Охранные условия (Guards):

- Условия, которые проверяются перед выполнением перехода. Обозначаются квадратными скобками, например: `[balance > 0]`.
- Пример: переход возможен только при соблюдении условия `[isLoggedIn == true]`.

### 4. Вложенные действия (Entry/Exit Actions):

- Обозначают действия, которые должны быть выполнены при входе или выходе из состояния. Они указываются внутри состояния, например, `entry / initialize()`, `exit / cleanup()`.

## Пример использования в GDD

Диаграммы состояний особенно полезны в **GDD** для описания поведения игровых объектов. Например, состояние персонажа может меняться в зависимости от взаимодействий с игроком или окружением:

- **Состояния персонажа:** «Idle», «Moving», «Attacking», «Dead».
- **Переходы:** Игрок может переводить персонажа из состояния «Idle» в «Moving» с помощью команды «`move()`», а из состояния «Moving» в «Attacking» — через «`attack()`».
- **События:** Враги могут атаковать персонажа, переводя его в состояние «Dead», если уровень здоровья достигает нуля.

## Пример для проектирования жизненного цикла объекта (SRS, SDD/TDD)

В **SRS** и **SDD** диаграммы состояний используются для документирования сложных объектов и процессов в системе:

- В **SRS** диаграммы состояний помогают определить **жизненный цикл объекта** (например, заказ клиента), который может проходить через состояния: «Created», «Processing», «Shipped», «Delivered».
- В **SDD** диаграммы состояний используются для описания алгоритмов или модулей, которые зависят от состояний (например, поведение системы при разных уровнях нагрузки).
- В **TDD** можно использовать диаграммы состояний для генерации тестов. Например, тестирование переходов системы между состояниями и проверка, что система реагирует на события корректно.

## Пример диаграммы состояний

Рассмотрим пример для **жизненного цикла заказа в интернет-магазине**:

### 1. Состояния:

- Created
- Processing
- Shipped
- Delivered

- Canceled

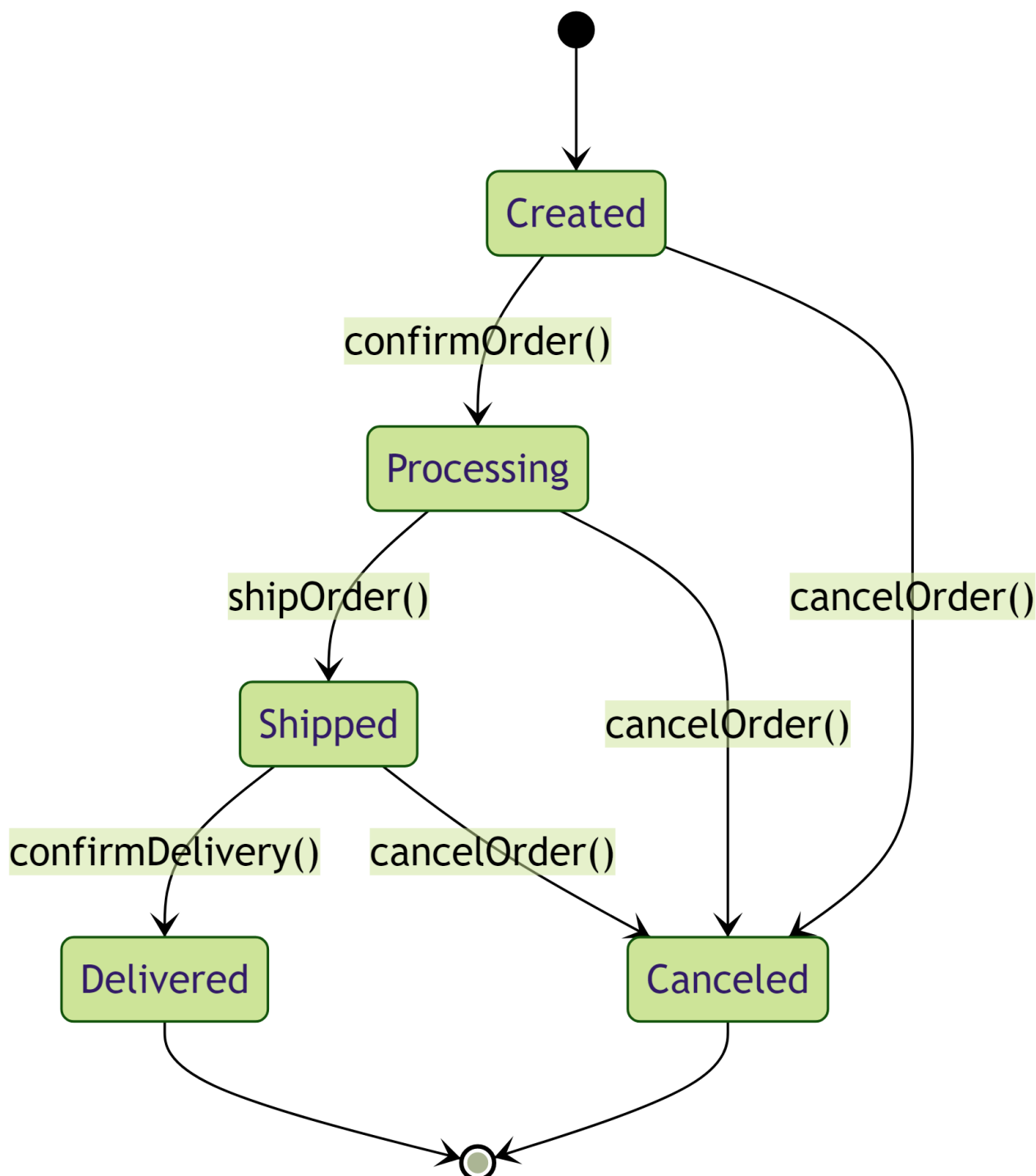
## 2. Переходы:

- Из «Created» в «Processing» при событии `confirmOrder()`.
- Из «Processing» в «Shipped» при событии `shipOrder()`.
- Из «Shipped» в «Delivered» при событии `confirmDelivery()`.
- В любое состояние может произойти переход в «Canceled» при событии `cancelOrder()`.

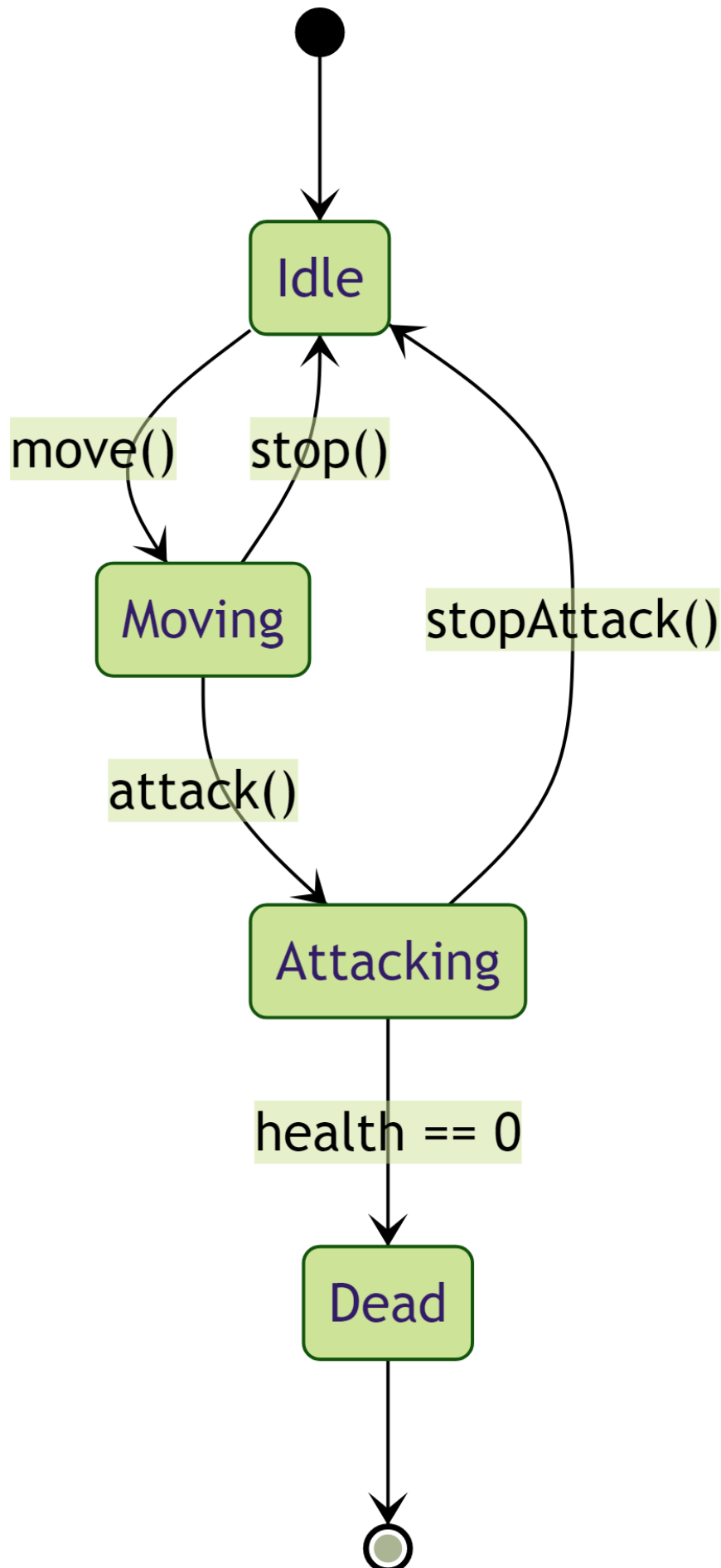
Такая диаграмма состояний помогает проектировать **поведение системы** на каждом этапе жизненного цикла заказа.

## Примеры диаграмм состояний

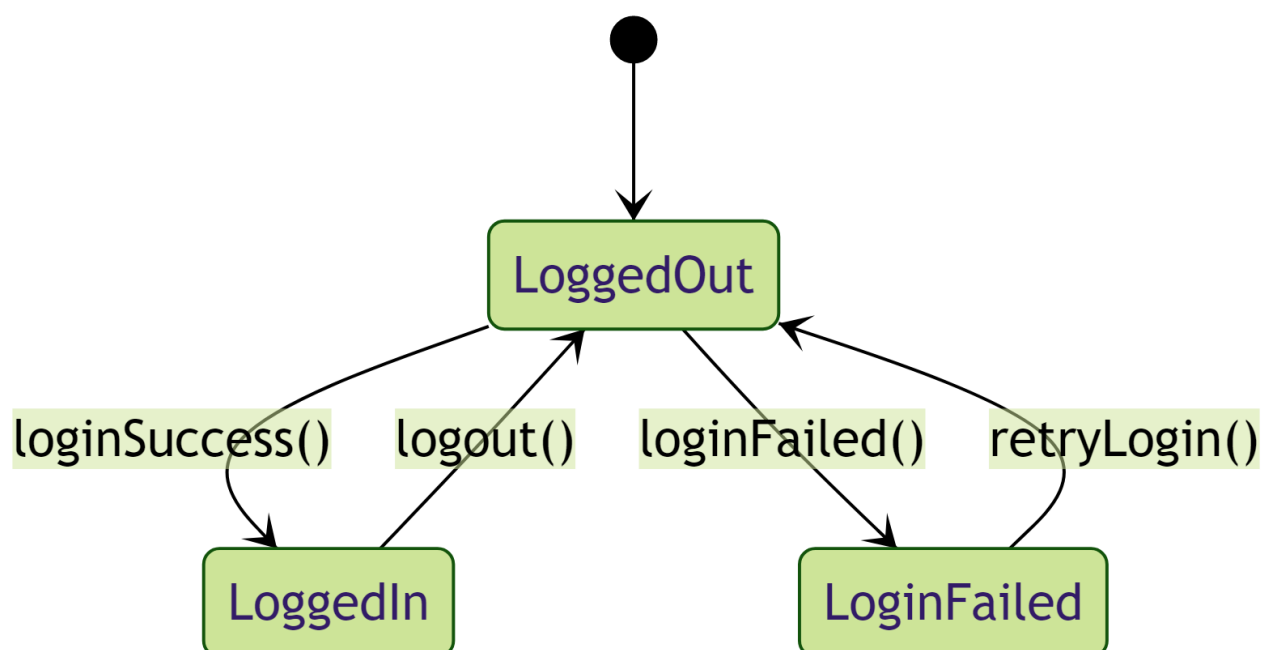
### 1. Жизненный цикл заказа в интернет-магазине



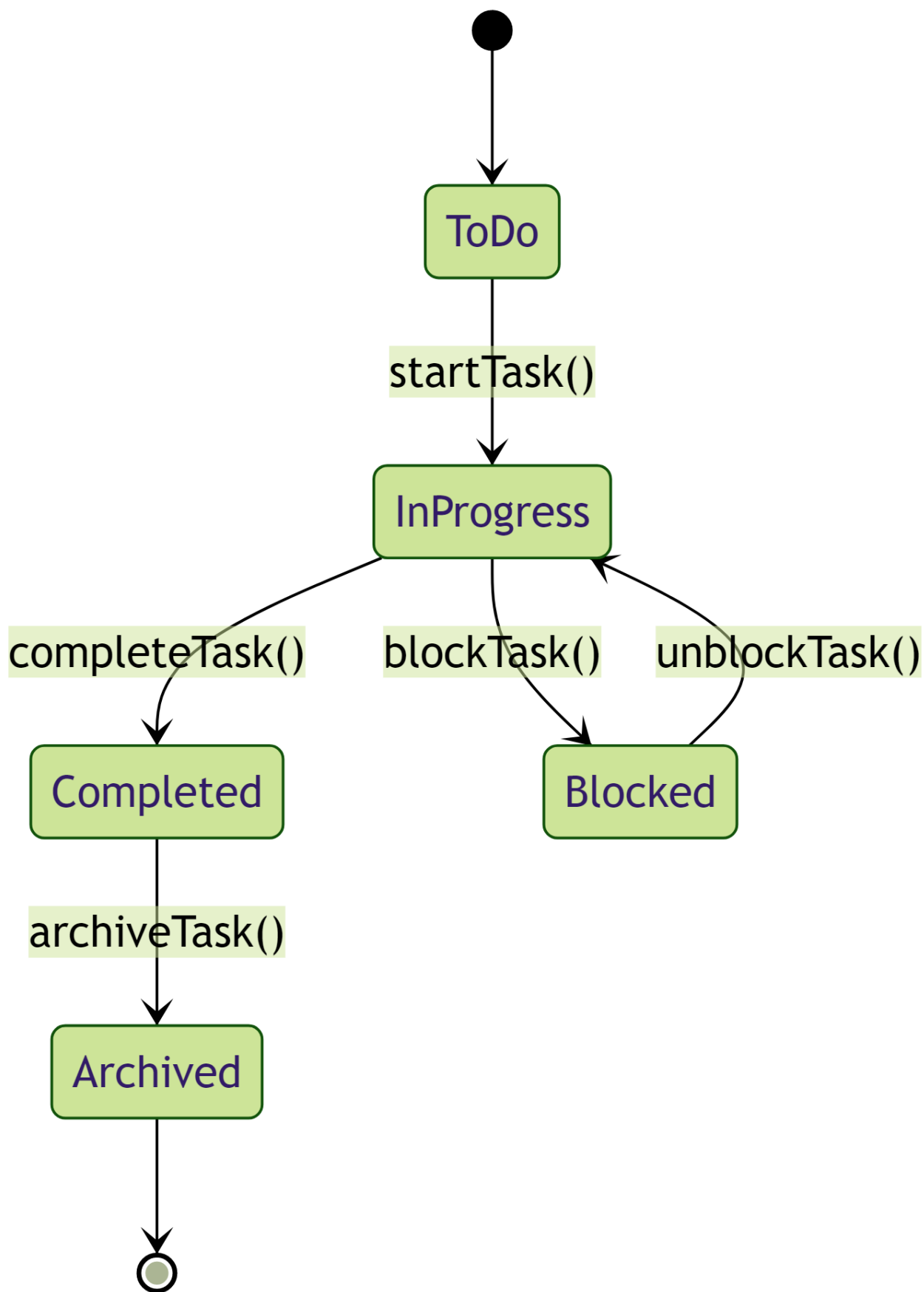
### 2. Состояния игрового персонажа



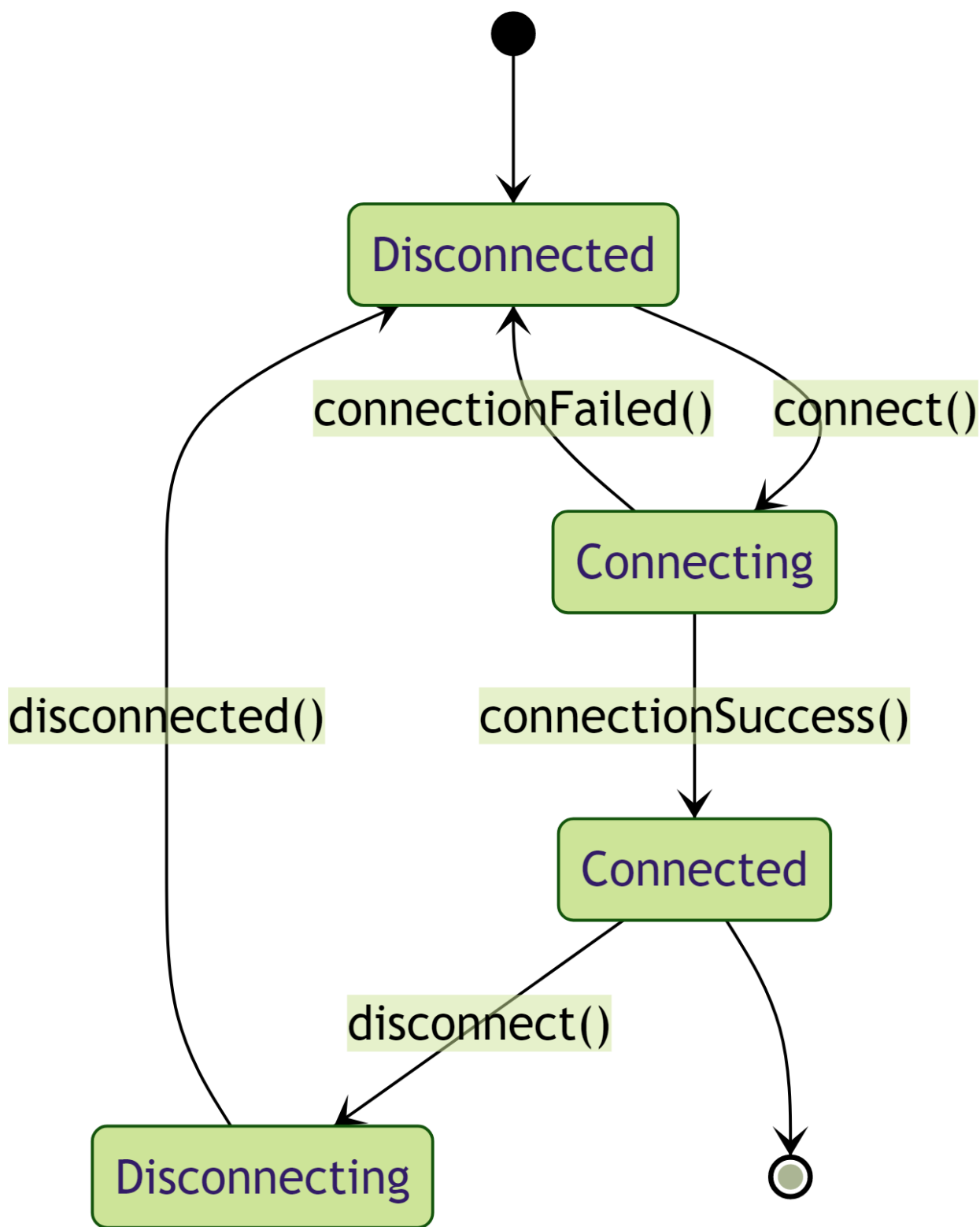
### 3. Процесс авторизации пользователя



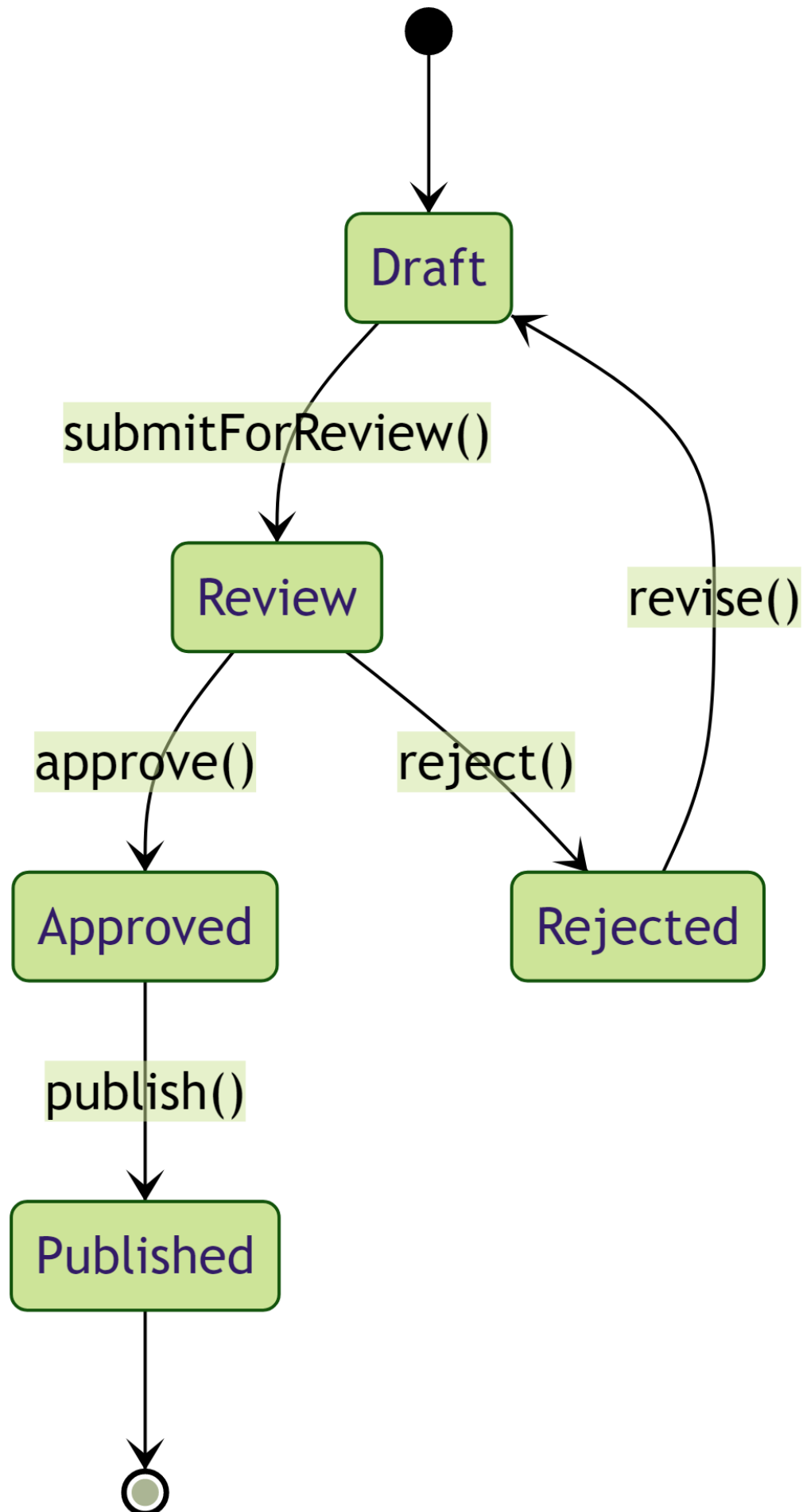
### 4. Жизненный цикл задачи в системе управления проектами



##### 5. Состояния сетевого подключения

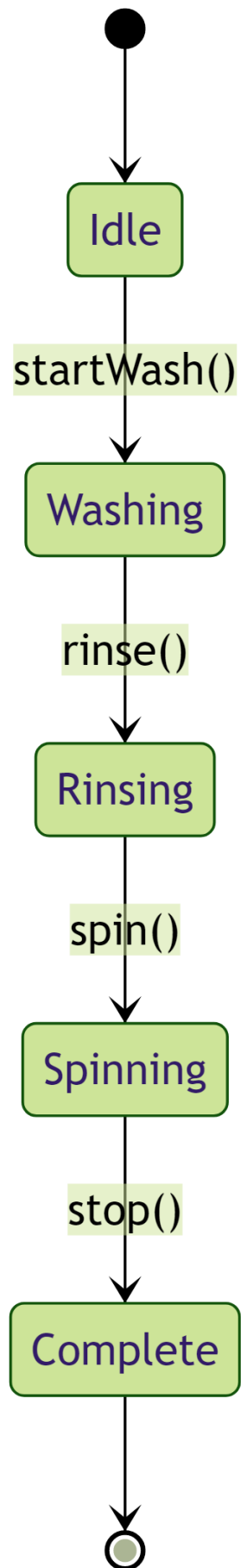


## 6. Состояния жизненного цикла документа



---

**7. Состояния бытового прибора (например, стиральной машины)**



Диаграммы состояний обеспечивают высокую степень детализации и точности при проектировании сложных систем. Они помогают понять и документировать, как объекты реагируют на события и как их поведение меняется со временем.

## Диаграммы конечных автоматов (State Machine Diagrams)

### Понятие конечных автоматов

Конечный автомат — это математическая модель, которая описывает поведение объекта как последовательность конечных состояний, через которые объект проходит в течение своего жизненного цикла. Каждое состояние связано с определённым набором событий или условий, которые вызывают переходы между состояниями. Конечные автоматы широко применяются для моделирования систем, где поведение объекта чётко определяется событиями или изменениями условий. В контексте программной инженерии конечные автоматы часто используются для описания систем с предсказуемым поведением, таких как коммуникационные протоколы, интерфейсы, игровые процессы и алгоритмы управления.

### Диаграммы конечных автоматов в UML

Диаграммы конечных автоматов (state machine diagrams) — это инструмент UML для визуализации изменения состояния объектов. Они отображают возможные состояния объекта и события, которые вызывают переходы между ними. В отличие от других диаграмм, диаграммы конечных автоматов акцентируются на **управлении состояниями объекта** и взаимосвязях между этими состояниями, что делает их полезными для **моделирования интерактивных систем** с чётко определёнными переходами.

### Основные элементы диаграммы конечных автоматов:

1. **Состояния (States)** — это различные состояния объекта, которые он может занимать. Состояния обозначаются прямоугольниками с закругленными углами, содержащими название состояния.
2. **Переходы (Transitions)** — показывают, как объект перемещается из одного состояния в другое в ответ на событие. Переходы изображаются стрелками между состояниями с подписью события или действия.
3. **Начальное состояние (Initial State)** — чёрный круг, обозначающий начальную точку жизненного цикла объекта.
4. **Конечное состояние (Final State)** — круг с точкой внутри, обозначающий завершение жизненного цикла объекта.
5. **События (Events)** — это триггеры, которые вызывают переходы между состояниями (например, пользовательские действия или внутренние сигналы системы).
6. **Охранные условия (Guard Conditions)** — логические выражения, которые проверяются перед выполнением перехода. Например, объект перейдет в новое состояние только при соблюдении определённого условия.

### Отличие от диаграмм состояний

Диаграммы конечных автоматов отличаются от диаграмм состояний тем, что они имеют более формализованную структуру и применяются, когда система строго управляется переходами и событиями. Они используются в системах с **чётко определёнными состояниями и строгими правилами переходов**. В то время как диаграммы состояний могут описывать общие процессы, конечные автоматы подходят для более строгих и формальных систем, таких как алгоритмы управления, где возможны определённые варианты взаимодействия с системой и строгие переходы между состояниями.

### Пример использования в приложениях

Конечные автоматы могут описывать сложные системы, такие как **игровые приложения**. Например, в игре каждый уровень может представлять отдельное состояние, а переходы между уровнями могут зависеть от выполнения

определённых условий (например, победа над врагами, сбор предметов и т.д.). Для персонажа в игре можно создать конечный автомат, который описывает такие состояния, как «ожидание», «движение», «атака» и «смерть».

### Пример:

- Состояния персонажа: **Idle** (ожидание), **Moving** (движение), **Attacking** (атака), **Dead** (смерть).
- Переходы между состояниями зависят от действий игрока (например, команда атаки переводит персонажа в состояние «Attacking»).

## Применение конечных автоматов в ИИ

Конечные автоматы широко применяются в области **искусственного интеллекта (ИИ)**, особенно в разработке игр и систем управления. Они позволяют моделировать поведение объектов и агентов, таких как NPC (non-player characters), где каждое состояние связано с конкретным набором действий или реакций на события в игровом мире. В ИИ конечные автоматы также применяются для построения алгоритмов, управляющих процессами, например, в робототехнике, где каждый робот может находиться в состоянии «ожидание», «движение», «выполнение задачи» и так далее.

## Инструменты для создания диаграмм конечных автоматов

Множество UML инструментов поддерживают создание диаграмм конечных автоматов, среди них: 1. **Lucidchart** — популярный онлайн-инструмент для создания UML диаграмм, поддерживающий диаграммы конечных автоматов и другие типы диаграмм. 2. **Visual Paradigm** — мощный инструмент для моделирования систем с поддержкой всех видов UML диаграмм, включая state machine diagrams. 3. **Miro** — онлайн-инструмент, который также поддерживает создание диаграмм конечных автоматов с возможностью совместной работы. 4. **Draw.io** — бесплатный и открытый инструмент для построения UML диаграмм, включая диаграммы состояний и конечных автоматов.

Ссылки на источники и инструменты:

- [Visual Paradigm](#)
- [Miro](#)
- [Restack](#)
- [PlantUML](#).
- [Draw.io](#)
- [Десктопный вариант Draw.io на Electron](#)
- [ALTOVA - UModel](#)
- [Astah UML](#)
- [StarUML](#)
- [Umbrello](#)
- [UMLet](#)
- [UMLetino - онлайн редактор](#)

## Диаграммы конечных автоматов и веб-интерфейсы

### Понятие конечных автоматов

Конечный автомат (Finite State Machine, FSM) — это модель, которая описывает объект, способный находиться в одном из конечного числа состояний, и переходить между ними в ответ на события или действия. Каждое состояние связано с определёнными действиями или условиями. FSM широко используются в компьютерных науках для управления поведением систем, особенно тех, где чётко определены правила переходов между состояниями.

## Применение конечных автоматов в веб-интерфейсах

Конечные автоматы активно применяются в проектировании **веб-интерфейсов**. Веб-приложения состоят из множества состояний, таких как «загрузка страницы», «ошибка», «успех» и прочие. FSM позволяет описывать, как веб-интерфейс должен реагировать на различные события, такие как клики пользователей, ошибки сети или успешные запросы.

Использование конечных автоматов в веб-интерфейсах помогает: 1. **Управлять состояниями UI**: Чёткое описание, как система переходит между состояниями, делает веб-интерфейсы более предсказуемыми и управляемыми. Например, состояние «Загрузка» может сменяться на «Успех» или «Ошибка» в зависимости от результата запроса. 2. **Обрабатывать параллельные действия**: FSM может описывать поведение интерфейса при одновременной загрузке данных, что часто используется в асинхронных веб-приложениях. 3. **Упрощать тестирование**: Диаграммы конечных автоматов позволяют легко моделировать различные сценарии поведения веб-приложения, что облегчает написание тестов.

## Пример использования в веб-приложениях

Конечные автоматы полезны в **моделировании поведения пользователя** в интерфейсе. Например, можно описать процесс авторизации пользователя с помощью конечного автомата: - Состояния: «Не авторизован», «Авторизация», «Авторизован». - Переходы: При успешной авторизации состояние меняется с «Не авторизован» на «Авторизован», при ошибке — возвращается в «Не авторизован».

Это упрощает обработку всех возможных состояний и сценариев, с которыми сталкивается пользователь при взаимодействии с интерфейсом.

## Отличия от диаграмм состояний

Диаграммы конечных автоматов в UML схожи с диаграммами состояний, но они более строгие и формализованные. Они акцентируют внимание на чётких переходах между состояниями и часто применяются в веб-приложениях для управления сложными интерактивными интерфейсами.

## Инструменты для создания диаграмм конечных автоматов

Существует множество UML-инструментов, поддерживающих диаграммы конечных автоматов: - **Lucidchart** — популярный инструмент для визуализации диаграмм, включая FSM. - **Visual Paradigm** — мощная платформа для создания UML-диаграмм с поддержкой FSM. - **XState** — библиотека JavaScript для управления состояниями в веб-приложениях, поддерживающая концепции конечных автоматов и их диаграммы.

## Небольшое отступление - YAKINDU Statechart Tools (itemis AG (company))

**YAKINDU Statechart Tools**, недавно переименованный в **itemis CREATE**, — это мощный инструмент для разработки конечных автоматов (Finite State Machines, FSM) и реактивных систем. Он используется для моделирования, симуляции и генерации исполнимых моделей конечных автоматов в различных областях, таких как программное обеспечение для встраиваемых систем, игры и сложные интерфейсы. Веб-разработчики и инженеры могут использовать этот инструмент для визуализации поведения систем с чёткими состояниями и переходами.

## Основные особенности YAKINDU:

1. **Моделирование конечных автоматов** — YAKINDU позволяет создавать сложные диаграммы состояний, которые можно графически редактировать и проверять на наличие ошибок.
2. **Симуляция** — Встроенный симулятор позволяет выполнять модели конечных автоматов и отслеживать поведение системы в реальном времени. Это упрощает отладку сложных систем, таких как веб-интерфейсы или игровые сценарии.

3. **Генерация кода** — Поддерживается автоматическая генерация кода на таких языках, как Java, C, C++ и Python, что делает YAKINDU отличным инструментом для интеграции конечных автоматов в готовые приложения.
4. **Поддержка многослойных автоматов** — Система поддерживает моделирование нескольких взаимодействующих конечных автоматов, что особенно полезно для сложных приложений, таких как игры или веб-сервисы.

## Применение в веб-интерфейсах:

Конечные автоматы играют важную роль в веб-интерфейсах, управляя состояниями UI. YAKINDU позволяет моделировать состояния компонентов интерфейса, такие как “Загрузка”, “Ошибка” или “Успех”, и легко описывать переходы между ними в ответ на действия пользователя или системные события. Это улучшает управление состояниями, особенно в **асинхронных веб-приложениях**, где требуется точное управление поведением интерфейса.

## Пример для игр:

В игровой индустрии конечные автоматы применяются для описания поведения игровых объектов или NPC (Non-Player Characters), которые могут находиться в таких состояниях, как «Ожидание», «Атака», «Отступление». YAKINDU предоставляет возможности для проектирования и тестирования таких моделей, а также генерации кода для их непосредственного использования в играх.

## Поддержка в ИИ:

YAKINDU также активно используется в разработке систем искусственного интеллекта (ИИ) для построения алгоритмов, основанных на конечных автоматах, таких как управление поведением агентов в играх или роботах.

Этот инструмент можно скачать и использовать для разработки как коммерческих, так и образовательных проектов с поддержкой бесплатных лицензий для студентов и исследователей.

Ссылки:

- [itemis CREATE Overview](#).

# Диаграммы компонентов и развертывания

## Диаграммы компонентов

**Диаграммы компонентов** в UML описывают **модульную структуру системы**, фокусируясь на организации и взаимосвязях между программными компонентами. Компоненты — это отдельные блоки, которые могут представлять целые подсистемы или модули. Эти диаграммы полезны на этапе проектирования и служат для документирования **архитектуры программного обеспечения**.

Основные элементы диаграммы компонентов: - **Компоненты**: представляют программные модули или блоки. Обычно это классы, библиотеки, пакеты или исполняемые файлы. - **Интерфейсы**: обозначаются как “леденцы” на диаграмме. Интерфейс — это набор операций, через которые компоненты взаимодействуют друг с другом. - **Соединения (Connectors)**: отображают зависимости между компонентами, показывая, как один компонент зависит от интерфейса другого компонента. Они могут быть выражены в виде ассоциаций или зависимостей.

Пример: В игровом приложении диаграмма компонентов может использоваться для отображения таких модулей, как игровой движок, физическая модель, графический интерфейс и взаимодействие с сервером.

Диаграммы компонентов используются в **SDD (Software Design Document)** для детализованного описания архитектуры системы. Они помогают понять, как организованы модули приложения и как они взаимодействуют друг с другом через интерфейсы.

## Диаграммы развертывания

**Диаграммы развертывания** (Deployment Diagrams) отображают **физическое развертывание системы**, фокусируясь на том, **где** программные компоненты будут работать и **на каких узлах** (аппаратных или программных). Эти диаграммы особенно важны для проектирования серверной инфраструктуры, в том числе в распределенных системах.

Основные элементы диаграммы развертывания: - **Узлы (Nodes)**: представляют собой физические или виртуальные машины (например, серверы, рабочие станции). Узлы обозначаются в виде 3D-прямоугольников. - **Артефакты (Artifacts)**: это файлы или данные, которые развертываются на узлах. Артефакты включают библиотеки, исполняемые файлы, базы данных. - **Ассоциации (Associations)**: линии между узлами обозначают каналы связи, по которым узлы взаимодействуют (например, сетевые протоколы или API).

Пример использования в играх: В игровом приложении диаграмма развертывания может использоваться для описания клиент-серверного взаимодействия, где клиент запускает игру, а сервер обрабатывает игровые данные и сохраняет результаты. Узлы могут включать **игровой сервер**, **базу данных игроков** и **клиентские машины**.

Диаграммы развертывания важны для **SDD/TDD** на этапе разработки инфраструктуры.

## Пример использования в игровой архитектуре

Игровое приложение, например, может включать такие узлы, как: - **Клиентская машина**, на которой работает игра. - **Игровой сервер**, обрабатывающий игровую логику и взаимодействие между игроками. - **Сервер базы данных**, где хранятся данные об игроках, такие как уровни и достижения.

Ещё раз: диаграммы развертывания отображает, как компоненты вашей развёрнутой (в облаке, на сервере(-ах),...) системы взаимодействуют между собой через сети или другие каналы связи.

## Применение UML в технической документации

UML — это универсальный язык моделирования, который широко используется в технической документации для описания структур и поведения систем. В зависимости от типа документации (GDD, SRS или SDD), используются различные диаграммы UML для отображения функциональных и архитектурных особенностей системы.

## Примеры использования UML-диаграмм в GDD, SRS и SDD

1. **GDD (Game Design Document)** — техническая документация для проектирования игр:

- **Диаграммы состояний**: используются для моделирования жизненного цикла игровых объектов, таких как персонажи или враги. Они показывают, как персонажи переходят из одного состояния в другое (например, из «Idle» в «Attacking»), и какие события или действия приводят к этим переходам.
- **Диаграммы конечных автоматов**: моделируют поведение игровых систем, таких как уровни сложности или поведение ИИ (например, NPC), которые зависят от конкретных событий и условий.
- **Диаграммы классов**: помогают описывать структуру игровых объектов, такие как персонажи, оружие или игровые элементы. Они отображают атрибуты и методы этих объектов и их взаимосвязи.
- **Диаграммы активности**: используются для описания процессов внутри игры, таких как логика выполнения действий персонажа, сценарии взаимодействия с объектами, или процесс выполнения миссии.

В **игровых проектах** UML помогает планировать и документировать сложные взаимодействия между персонажами, игровыми уровнями, а также моделировать поведение и управление состояниями игровых объектов.

2. **SRS (Software Requirements Specification)** — документ, описывающий функциональные и нефункциональные требования к системе:

- **Диаграммы прецедентов (Use Case Diagrams)**: моделируют взаимодействие пользователей с системой. Например, в системе управления игрой это может быть сценарий взаимодействия игрока с интерфейсом, регистрация или начало уровня.

- **Диаграммы классов:** описывают ключевые сущности и их отношения, например, моделирование игровых объектов и их атрибутов в SRS может отображать структуру игровых персонажей и взаимодействие между ними.
- **Диаграммы активности:** используются для описания рабочих процессов, таких как сценарии использования игроками различных функций игры, например, процесс прохождения уровня или завершение миссий.

В **SRS документах** UML диаграммы помогают уточнить функциональные требования, определяя взаимодействие пользователей с системой через прецеденты и модели поведения.

### 3. **SDD (Software Design Document)** — детальная документация, описывающая архитектуру системы:

- **Диаграммы классов:** используются для проектирования структуры системы, показывая внутренние классы и их взаимосвязи. Например, можно описать, как игровые компоненты, такие как «Игрок», «NPC», «Оружие», взаимодействуют друг с другом на уровне кода.
- **Диаграммы компонентов:** помогают описывать модульную структуру игрового приложения, отображая, как различные компоненты системы (например, игровой движок, графическая подсистема, звуковая подсистема) взаимодействуют между собой.
- **Диаграммы последовательности:** важны для описания взаимодействий между объектами во времени, например, обмен сообщениями между клиентом и сервером в многопользовательской игре.
- **Диаграммы конечных автоматов:** используются для описания поведения сложных систем в игре, таких как ИИ или управление уровнями сложности.

В **SDD** диаграммы UML служат для детализированного описания архитектуры системы, показывая, как игровые компоненты взаимодействуют между собой на уровне кода и инфраструктуры.

## Практическое использование UML при разработке приложений

UML широко используется при разработке сложных проектов игровых приложений, в корпоративных, мобильных и веб-системах. Он помогает **визуализировать** сложные структуры и процессы, улучшать **коммуникацию между членами команды** и создавать единую архитектурную картину системы. Благодаря поддержке таких инструментов, как Lucidchart, Visual Paradigm, Astah, UMLet, MS Visio, etc. разработчики могут быстро создавать и симулировать диаграммы, проверяя работоспособность системы ещё на этапе проектирования.

**Кроме этого**, фрагменты UML диаграмм можно использовать в ежедневном **общении** на **митингах**, как элементы **единого языка** среди коллег по проекту. Например - у общей доски с маркером.

В игровых приложениях UML применяется для моделирования таких аспектов, как:

- **Поведение игровых объектов** и их взаимодействие с окружением.
- **Моделирование ИИ** для NPC и игровых сценариев.
- **Оптимизация клиент-серверного взаимодействия** в многопользовательских играх.

Ссылки:

- [Lucidchart — UML Diagrams](#)
- [Visual Paradigm — UML Diagrams](#).